# Rockchip RT-Thread USB 开发指南

文件标识：RK-KF-YF-106

发布版本：V1.6.0

日期：2026-02-13

文件密级：□绝密 □秘密 □内部资料 ■公开

**免责声明**

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

**商标声明**

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

**版权所有 © 2026 瑞芯微电子股份有限公司**

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

**前言**

**概述**

本文介绍了 Rockchip RT-Thread SDK 的 USB 支持情况，说明 USB 代码框架和配置方法，以及 USB Device 和 Host 使用示例。

**产品版本**

| 芯片名称 | RT-Thread 版本 |
| --- | --- |
| RK1808、RK2108、RK2206、RK3308、RK3358、RK3568、RK625 | RT-Thread 3.1.x 及以上版本 |
| RK2116、RK2118、RK3506 | RT-Thread 4.1.x |
| RK1820 | RT-Thread 5.1.x |

**读者对象**

本文档（本指南）主要适用于以下工程师：

技术支持工程师
软件开发工程师

**修订记录**

| 版本号 | 作者 | 修改日期 | 修改说明 |
| --- | --- | --- | --- |
| V1.0.0 | 吴良峰 | 2019-07-23 | 初始版本 |
| V1.0.1 | 陈谋春 | 2020-03-17 | 修正链接 |
| V1.0.2 | 吴良峰 | 2020-05-25 | 修正格式 |
| V1.1.0 | 王明成 | 2021-03-12 | 新增 UVC 使用示例 |
| V1.2.0 | 吴良峰、郑见炜 | 2024-06-11 | 新增 RK2118 RT-Thread 4.1.x 说明<br>新增 Device WINUSB/CDC_VCOM 使用示例<br>完善 Device HID/MSC 使用示例<br>完善 Device ADB 使用示例，支持 ADB push/pull/shell 功能<br>新增 Device 控制器的端点硬件 FIFO 配置说明<br>新增 Device VBUS Detect 软硬件使用说明<br>新增 Device speed 配置说明<br>新增 Host MSC 使用示例<br>完善 Host 配置说明 |
| V1.3.0 | 吴良峰 | 2024-09-23 | 新增 RT-Thread 4.1.x Device UAC 复合设备的使用方法<br>新增 RT-Thread 4.1.x Multi Host 的配置方法<br>新增 RT-Thread 4.1.x RK3506 支持说明 |
| V1.4.0 | 吴良峰 | 2025-08-01 | 重构章节，完善 USB 配置说明，提高可读性<br>新增 HAL USB 代码路径及函数接口说明<br>新增 RT-Thread 5.1.x USB 协议栈说明<br>新增 RK1820 支持说明<br>新增 RK2116 支持说明 |
| V1.5.0 | 吴良峰 | 2025-09-17 | 完善 RT-Thread 4.1.x Device UAC 使用示例<br>完善 RT-Thread 5.1.x Host Udisk 使用示例 |
| V1.6.0 | 吴良峰 | 2026-02-13 | 完善 RT-Thread 3.1.x/4.1.x Device UAC1 使用示例<br>新增支持 RT-Thread 3.1.x/4.1.x Device UAC2 使用示例<br>新增支持 RT-Thread 5.1.x Device CDC_VCOM 使用示例<br>新增支持 RT-Thread 5.1.x Device WINUSB&CDC_ACM 使用示例 |

**目录**

# Rockchip RT-Thread USB 开发指南


1. Rockchip RT-Thread USB 简介
2. USB 代码框架介绍
3. USB 代码路径
   - 3.1 HAL USB 代码路径
     - 3.1.1 HAL USB2.0 OTG 代码路径
     - 3.1.2 HAL USB2.0 OTG 函数接口说明
     - 3.1.3 HAL USB2.0 Host 代码路径
     - 3.1.4 HAL USB2.0 Host 函数接口说明
     - 3.1.5 HAL USB2.0/3.0 DRD 代码路径
     - 3.1.6 HAL USB2.0/3.0 DRD 函数接口说明
   - 3.2 RT-Thread USB 组件代码路径
     - 3.2.1 RT-Thread 3.1.x/4.1.x USB 组件代码路径
     - 3.2.2 RT-Thread 5.1.x USB 组件代码路径
   - 3.3 RT-Thread USB 适配层代码路径
     - 3.3.1 RT-Thread 3.1.x USB 驱动适配层的代码路径
     - 3.3.2 RT-Thread 4.1.x USB 驱动适配层的代码路径
     - 3.3.3 RT-Thread 5.1.x USB 适配层的代码路径
4. USB 配置
   - 4.1 USB Device 配置
     - 4.1.1 BSP USB Device 配置
       - 4.1.1.1 USB Device VBUS Detect 使用说明
       - 4.1.1.2 USB Device speed 配置说明
       - 4.1.1.3 USB Device 端点 FIFO 配置说明
     - 4.1.2 RT-Thread 3.1.x/4.1.x USB Device 配置
     - 4.1.3 RT-Thread 5.1.x USB Device 配置
   - 4.2 USB Host 配置
     - 4.2.1 BSP USB Host 配置
       - 4.2.1.1 USB Host VBUS 控制说明
     - 4.2.2 RT-Thread 3.1.x/4.1.x USB Host 配置
     - 4.2.3 RT-Thread 5.1.x USB Host 配置
5. USB 设备管理接口
6. USB 内存管理接口
7. USB Device 使用示例
   - 7.1 RT-Thread 3.1.x/4.1.x USB Device 使用示例
     - 7.1.1 USB Device 创建和注册
     - 7.1.2 USB Device HID 使用示例
     - 7.1.3 USB Device ADB 使用示例
     - 7.1.4 USB Device MSC 使用示例
     - 7.1.5 USB Device UAC1 使用示例
     - 7.1.6 USB Device UAC2 使用示例
     - 7.1.7 USB Device UVC 使用示例
     - 7.1.8 USB Device WINUSB 使用示例
     - 7.1.9 USB Device CDC_VCOM 使用示例
   - 7.2 RT-Thread 5.1.x USB Device 使用示例
     - 7.2.1 USB Device ADB 使用示例
     - 7.2.2 USB Device WINUSB 使用示例
     - 7.2.3 USB Device CDC_ACM 使用示例
     - 7.2.4 USB Device CDC_VCOM 使用示例
       - 7.2.4.1 USB Virtual Console切换方式
     - 7.2.5 USB Device WINUSB&CDC_ACM 复合设备使用示例
8. USB Host 使用示例
   - 8.1 RT-Thread 3.1.x/4.1.x USB Host 使用示例
     - 8.1.1 USB Host 创建和注册

8.1.2 USB Host MSC 使用示例
8.2 RT-Thread 5.1.x USB Host 使用示例
8.2.1 USB Host MSC 使用示例

# 1. Rockchip RT-Thread USB 简介

- 支持 RT-Thread 3.1.x 及以上版本 USB 组件功能
- RT-Thread USB 组件实现了 USB Device/Host 协议栈以及各种 USB 设备类驱动
- RT-Thread 5.1.x 版本支持 RTT 官方 USB 组件和 CherryUSB 组件
- 所有 USB 控制器使用 DMA 传输模式

表 1 Rockchip RT-Thread USB 支持情况

| 芯片 | 支持 USB 接口 | 支持 RTT 版本 | 支持 USB 组件 |
|---|---|---|---|
| RK1808 | 1 * USB2.0 HOST | RTT 3.1.x/4.1.x | RTT USB 组件 |
| RK1820 | 1 * USB3.0 DRD + 1 * USB2.0 DRD | RTT 5.1.x | CherryUSB 组件 |
| RK2108 | 1 * USB2.0 Device | RTT 3.1.x/4.1.x | RTT USB 组件 |
| RK2116 | 1 * USB2.0 OTG | RTT 4.1.x | RTT USB 组件 |
| RK2118 | 1 * USB2.0 OTG | RTT 4.1.x | RTT USB 组件 |
| RK2206 | 1 * USB2.0 OTG | RTT 3.1.x | RTT USB 组件 |
| RK3308 | 1 * USB2.0 OTG + 1 * USB2.0 HOST | RTT 3.1.x/4.1.x | RTT USB 组件 |
| RK3358 | 1 * USB2.0 OTG + 1 * USB2.0 HOST | RTT 3.1.x/4.1.x | RTT USB 组件 |
| RK3506 | 2 * USB2.0 OTG | RTT 4.1.x | RTT USB 组件 |
| RK3568 | 2 * USB2.0 HOST | RTT 3.1.x/4.1.x | RTT USB 组件 |
| RK625 | 1 * USB2.0 OTG | RTT 3.1.x | RTT USB 组件 |

*Note*

1. USB2.0 OTG 和 USB3.0 DRD 接口的 Device/Host 模式切换，目前仅支持通过 menuconfig 进行静态配置；
2. USB 接口的详细特性，请参考 Rockchip SDK 文档《Rockchip_Developer_Guide_USB_CN》。

# 2. USB 代码框架介绍

USB 框架是基于 RT-Thread 提供的 I/O 设备模型框架 实现的，如下图 1 所示，从上到下分别是 I/O 设备管理层、RTT USB 组件、RTT 与 HAL 的适配层、HAL 层 USB 控制器驱动。




图 1 RT-Thread USB 框架图

- 应用程序通过 I/O 设备管理接口来访问 USB 设备类驱动，然后通过 USB 设备类驱动与底层的USB 控制器进行数据（或控制）交互
- I/O 设备管理层实现了对 USB 设备类驱动程序的封装，提供标准接口给应用程序
- RTT USB 组件实现了 USB Device/Host 协议栈以及各种 USB 设备类驱动。USB 设备类驱动通过 USB 协议栈（USB Core）提供的标准接口与底层的 USB 控制器进行数据交互。USB 协议栈通过适配层提供的标准接口与 USB 控制器进行数据（或控制）交互
- RTT 与 HAL 的适配层，作用是配置 Rockchip USB 控制器的硬件信息，并通过 HAL 层 USB 控制器驱动提供的接口初始化 USB 控制器。同时，根据 RTT USB 组件定义的标准接口，将 HAL 层 USB 控制器驱动抽象为方便 RTT USB 组件直接访问的接口函数
- HAL 层 USB 控制器驱动实现了对 USB 控制器进行读写操作的底层代码，可以适配不同的 RTOS

# 3. USB 代码路径

## 3.1 HAL USB 代码路径

### 3.1.1 HAL USB2.0 OTG 代码路径

```
lib/hal/
├── inc
│   ├── hal_hcd.h      /* USB2.0 OTG Host 驱动头文件 */
│   ├── hal_pcd_ex.h   /* USB2.0 OTG Device 扩展功能头文件 */
│   ├── hal_pcd.h      /* USB2.0 OTG Device 驱动头文件 */
│   ├── hal_usb_core.h /* USB2.0 OTG Core 驱动头文件 */
│   └── hal_usb_phy.h  /* USB2.0 PHY 驱动头文件 */
└── src
    ├── hal_hcd.c      /* USB2.0 OTG Host 驱动 */
    ├── hal_pcd.c      /* USB2.0 OTG Device 驱动 */
    ├── hal_pcd_ex.c   /* USB2.0 OTG Device 扩展功能驱动 */
    ├── hal_usb_core.c /* USB2.0 OTG Core 驱动 */
    └── hal_usb_phy.c  /* USB2.0 PHY 驱动 */
```

### 3.1.2 HAL USB2.0 OTG 函数接口说明

HAL USB2.0 OTG 模块的函数接口及使用方法，可以用 doxygen 生成的文档，并查看对应的 HCD/PCD/PCDEx/USB_CORE/USB_PHY 等模块章节。

以 RK2118 HAL USB2.0 OTG 模块为例，用 doxygen 生成文档的命令如下：

`hal$ cat tools/Doxyfile tools/doxyfile_rk2118 | doxygen -`

生成的文档位于 `hal/html/index.html`

以 HCD HAL module driver 使用为例，参考如下：

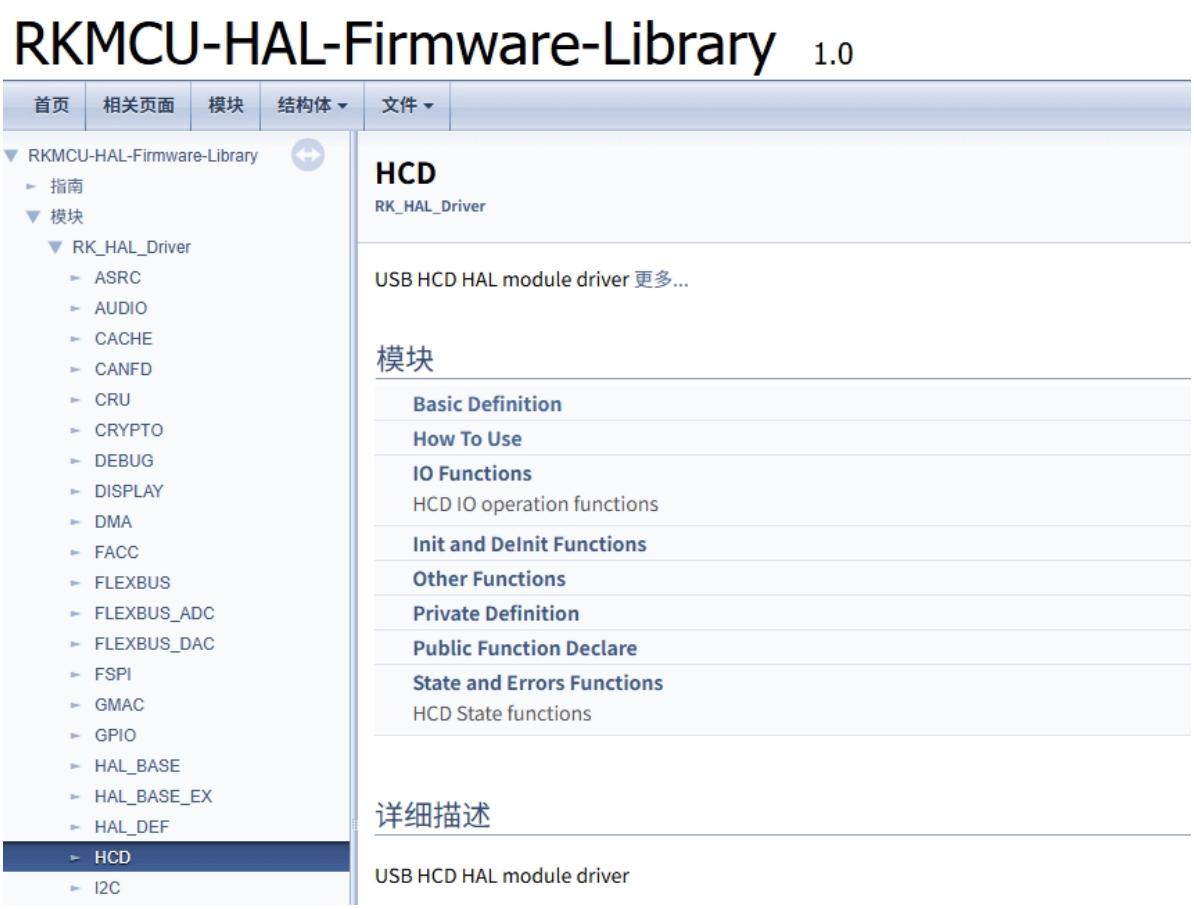



图 2 HAL USB2.0 OTG HCD 模块使用示例

### 3.1.3 HAL USB2.0 Host 代码路径

```
lib/hal/
├── inc
│   ├── hal_ehci.h    /* USB2.0 Host EHCI 控制器驱动头文件 */
│   ├── hal_ohci.h    /* USB1.1&1.0 Host OHCI 控制器驱动头文件 */
│   ├── hal_usbh.h    /* USB2.0 Host Core 驱动头文件 */
│   └── hal_usb_phy.h /* USB2.0 PHY 驱动头文件 */
└── src
    ├── hal_ehci.c     /* USB2.0 Host EHCI 控制器驱动 */
    ├── hal_ehci_iso.c /* USB2.0 Host EHCI 控制器同步传输驱动 */
    ├── hal_ohci.c     /* USB1.1&1.0 Host OHCI 控制器驱动 */
    ├── hal_usbh.c     /* USB2.0 Host Core 驱动 */
    └── hal_usb_phy.c  /* USB2.0 PHY 驱动 */
```

### 3.1.4 HAL USB2.0 Host 函数接口说明

HAL USB2.0 Host 模块的函数接口及使用方法，可以用 doxygen 生成的文档，并查看对应的 EHCI/OHCI/USB_HOST/USB_PHY 等模块章节。

以 RK3568 HAL USB2.0 Host 模块为例，用 doxygen 生成文档的命令如下：

```
hal$ cat tools/Doxyfile tools/doxyfile_rk3568 | doxygen -
```

生成的文档位于 `hal/html/index.html`

以 EHCI HAL module driver 使用为例，参考如下：




图 3 HAL USB2.0 EHCI 模块使用示例

### 3.1.5 HAL USB2.0/3.0 DRD 代码路径

```
lib/hal/
├── inc
│   ├── hal_drd_core.h   /* USB DRD Core 驱动头文件 */
│   ├── hal_drd_gadget.h /* USB DRD Gadget 驱动头文件 */
│   ├── hal_xhci.h       /* USB DRD Host xHCI 驱动头文件 */
│   └── hal_usb_phy.h    /* USB2.0&3.0 PHY 驱动头文件 */
└── src
    ├── hal_drd_core.c   /* USB DRD Core 驱动 */
    ├── hal_drd_gadget.c /* USB DRD Gadget 驱动 */
    ├── hal_usb_phy.c    /* USB2.0&3.0 PHY 驱动 */
    └── hal_xhci.c       /* USB DRD Host xHCI 驱动 */
```

### 3.1.6 HAL USB2.0/3.0 DRD 函数接口说明

HAL USB2.0/3.0 DRD 模块的函数接口及使用方法，可以用 doxygen 生成的文档，并查看对应的 USB_DRD_CORE/USB_DRD_GADGET/USB_PHY/USB_XHCI 等模块章节。

以 RK1820 HAL USB DRD 模块为例，用 doxygen 生成文档的命令如下：

```
hal$ cat tools/Doxyfile tools/doxyfile_rk1820 | doxygen -
```

生成的文档位于 `hal/html/index.html`

以 HAL USB_DRD_GADGET module driver 使用为例，参考如下：

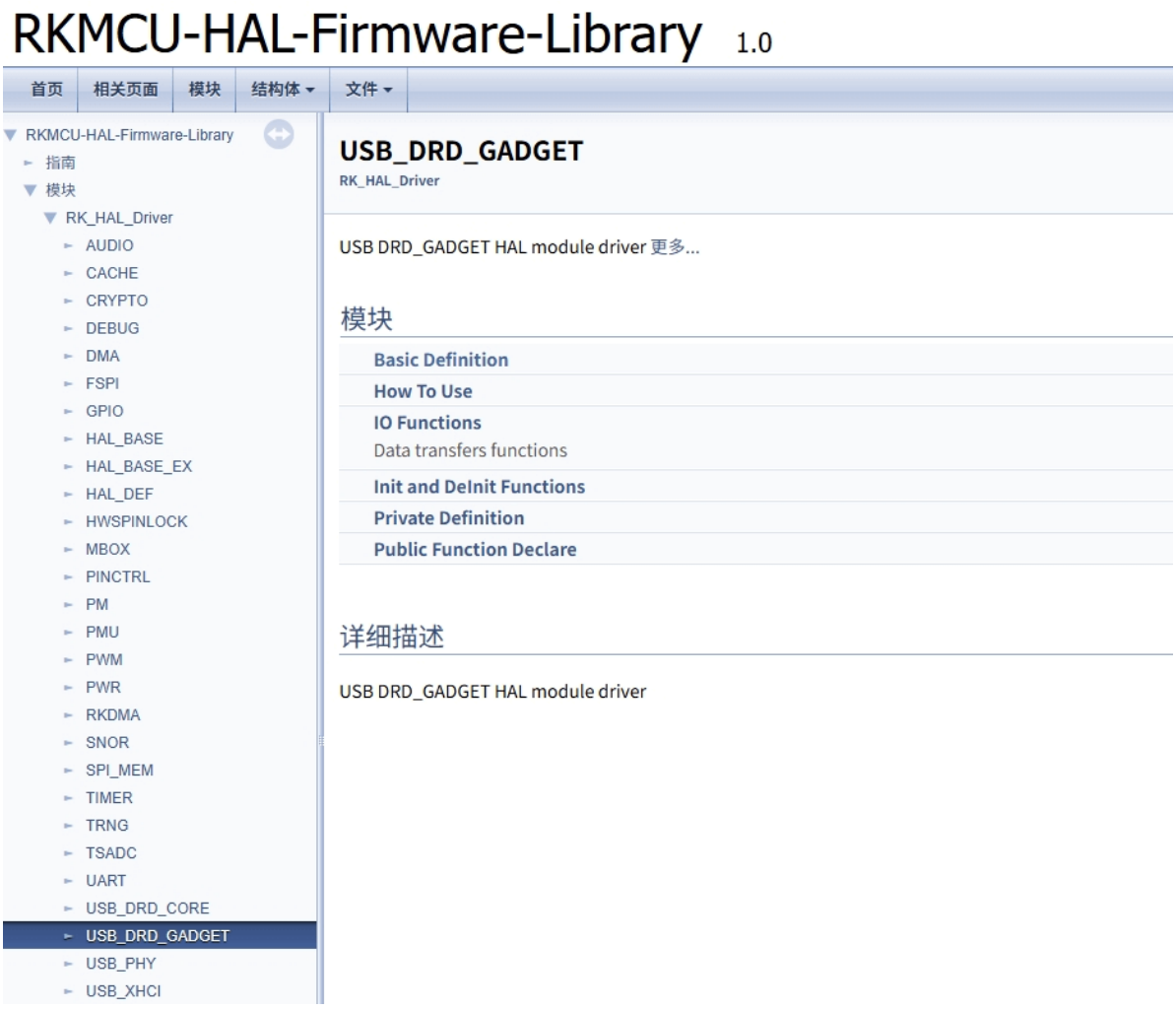


图 4 HAL USB_DRD_GADGET 模块使用示例

## 3.2 RT-Thread USB 组件代码路径

### 3.2.1 RT-Thread 3.1.x/4.1.x USB 组件代码路径

RT-Thread USB 组件位于 `rt-thread/components/drivers/usb`，是 RT-Thread 依据 USB2.0 协议规范将 USB 协议栈逻辑高度抽象，支持 Host（主机）模式和 Device（从机）模式。该组件在驱动移植方面提供了友好的移植接口，开发者可将厂商 BSP 中的 HCD（Host Controller Driver）和 PCD（Peripheral Controller Driver）驱动代码直接接入到 RT-Thread。

USB 组件的框架代码路径如下：

```
rt-thread$ tree components/drivers/usb
components/drivers/usb
├── SConscript
├── usbdevice
│   ├── class  /* USB Device 设备类驱动 */
│   │   ├── audio_mic.c
│   │   ├── audio_speaker.c
│   │   ├── cdc.h
│   │   ├── cdc_vcom.c
│   │   ├── ecm.c
│   │   ├── hid.c
│   │   ├── hid.h
│   │   ├── mstorage.c
│   │   ├── mstorage.h
│   │   ├── ndis.h
│   │   ├── rndis.c
│   │   ├── rndis.h
│   │   ├── uac1.c
│   │   ├── uac1.h
│   │   ├── uaudioreg.h
│   │   ├── winusb.c
│   │   └── winusb.h
│   ├── core  /* USB Device 核心代码，实现 Device 协议栈 */
│   │   ├── usbdevice.c
│   │   └── usbdevice_core.c
│   └── SConscript
└── usbhost
    ├── class /* USB Host 设备类驱动 */
    │   ├── adk.c
    │   ├── adk.h
    │   ├── hid.c
    │   ├── hid.h
    │   ├── mass.c
    │   ├── mass.h
    │   ├── udisk.c
    │   ├── ukbd.c
    │   └── umouse.c
    ├── core  /* USB Host 核心代码，实现 Host 协议栈 */
    │   ├── driver.c
```

```
    │   ├── hub.c
    │   ├── usbhost.c
    │   └── usbhost_core.c
    └── SConscript
```

*Note*

- RT-Thread 3.1.x/4.1.x USB 组件 USB Device Class 支持情况

1. CDC-ACM（Communication Device Class - Abstract Control Model，USB 虚拟串口）
2. CDC-ECM（Communication Device Class - Ethernet Networking Control Model，USB 以太网控制模型）
3. HID（Human Interface Device, 人体学输入设备，支持 Keyboard/Mouse/General HID/Media keyboard）
4. MStorage（Mass Storage，大容量存储设备）
5. RNDIS（Remote Network Driver Interface Specification，远程网络驱动接口）
6. WINUSB（Windows USB, 微软通用 USB）

- RT-Thread 3.1.x/4.1.x USB 组件 USB Host Class 支持情况

1. MStorage（Mass Storage，大容量存储设备）
2. HUB（基础驱动已支持，功能在调试中）
3. HID（基础驱动已支持，功能在调试中）

### 3.2.2 RT-Thread 5.1.x USB 组件代码路径

RT-Thread 5.1.x 支持两种 USB 组件：

1. RTT Legacy USB 组件，与 RT-Thread 3.1.x/4.1.x USB 组件 一样。
2. CherryUSB 组件

```
rt-thread$ tree -d components/drivers/usb/cherryusb
components/drivers/usb/cherryusb
├── class   /* usb class 类主从驱动 */
│   ├── adb
│   ├── aoa
│   ├── audio
│   ├── cdc
│   ├── dfu
│   ├── hid
│   ├── hub
│   ├── midi
│   ├── msc
│   ├── template
│   ├── vendor
│   │   ├── net
│   │   ├── serial
│   │   ├── wifi
│   │   └── xbox
│   ├── video
│   └── wireless
├── common  /* usb spec 定义、常用宏、标准接口定义 */
├── core    /* usb 主从协议栈核心实现 */
├── demo    /* 主从 class demo */
```

```
│   ├── adb
│   └── bootuf2
├── osal    /* os 封装层 */
└── port    /* usb 主从需要实现的 porting 接口 */
    ├── aic
    ├── bouffalolab
    ├── ch32
    ├── chipidea
    ├── dwc2
    ├── ehci
    ├── fsdev
    ├── hpm
    ├── kinetis
    ├── musb
    ├── nrf5x
    ├── ohci
    ├── pusb2
    │   └── rt-thread
    ├── rp2040
    ├── template
    └── xhci
        └── phytium
            └── rt-thread

/* cherryusb 说明文档 */
rt-thread$ components/drivers/usb/cherryusb/README_zh.md

/* cherryusb 板级 usb config 文件，以 RK1820 SDK 为例 */
rt-thread$ bsp/rockchip/rk1820/board/common/usb_config.h
```

*Note*

1. RT-Thread 5.1.x 建议优先使用 CherryUSB 组件进行 USB 功能开发；
2. CherryUSB 源码下载：https://github.com/cherry-embedded/CherryUSB
3. CherryUSB 使用指南：https://cherryusb.readthedocs.io/zh-cn/latest/index.html
4. Rockchip SDK USB ADB 驱动不使用 Cherryusb 组件中的 class ADB 驱动，而是使用 Rockchip 自己开发的 ADB 驱动，源码在：

   `rt-thread/third_party/adbd`
5. Rockchip SDK 不使用 CherryUSB 组件中的 port 接口驱动，而是使用 HAL 仓库 USB 驱动

## 3.3 RT-Thread USB 适配层代码路径

### 3.3.1 RT-Thread 3.1.x USB 驱动适配层的代码路径

```
rt-thread$ tree bsp/rockchip/common/drivers/
bsp/rockchip/common/drivers/
├── drv_usbd.c     /* RT-Thread 与 HAL USB2.0 OTG Device 控制器驱动之间的适配层 */
├── drv_usbd.h     /* RT-Thread 与 HAL USB2.0 OTG Device 控制器驱动之间的适配层头文件
*/
├── drv_usbhost.c  /* RT-Thread 与 HAL USB2.0 Host EHCI&OHCI 控制器驱动之间的适配层
*/
└── drv_usbh.c     /* RT-Thread 与 HAL USB2.0 OTG Host 控制器驱动之间的适配层 */
```

### 3.3.2 RT-Thread 4.1.x USB 驱动适配层的代码路径

```
rt-thread$ tree bsp/rockchip/common/drivers/
bsp/rockchip/common/drivers/
├── drv_usbd.c     /* RT-Thread 与 HAL USB2.0 OTG Device 控制器驱动之间的适配层 */
├── drv_usbd.h     /* RT-Thread 与 HAL USB2.0 OTG Device 控制器驱动之间的适配层头文件
*/
├── drv_usbhost.c  /* RT-Thread 与 HAL USB2.0 Host EHCI&OHCI 控制器驱动之间的适配层
*/
└── drv_usbotgh.c  /* RT-Thread 与 HAL USB2.0 OTG Host 控制器驱动之间的适配层 */
```

### 3.3.3 RT-Thread 5.1.x USB 适配层的代码路径

```
rt-thread$ tree bsp/rockchip/common/drivers
bsp/rockchip/common/drivers
├── cherryusb
│   ├── cherryusb_dc_drd.c  /* cherryusb 与 HAL USB DRD Gadget 控制器驱动之间的适配层
*/
│   ├── cherryusb_hc_xhci.c /* cherryusb 与 HAL USB DRD xHCI 控制器驱动之间的适配层
*/
│   ├── cherryusb_mem.c     /* cherryusb 提供给 HAL USB 驱动的内存管理 */
│   └── SConscript
├── drv_usbd.c    /* RTT Legacy USB 组件与 HAL USB2.0 OTG Device 控制器驱动之间的适配
层 */
├── drv_usbd.h    /* RTT Legacy USB 组件与 HAL USB2.0 OTG Device 控制器驱动之间的适配
层头文件 */
├── drv_usbhost.c /* RTT Legacy USB 组件与 HAL USB2.0 Host EHCI&OHCI 控制器驱动之间的
适配层 */
└── drv_usbotgh   /* RTT Legacy USB 组件与 HAL USB2.0 OTG Host 控制器驱动之间的适配层
*/
```

## 4. USB 配置

## 4.1 USB Device 配置

### 4.1.1 BSP USB Device 配置

USB2.0 OTG Device BSP 参考配置如下：

```
#ifdef HAL_PCD_MODULE_ENABLED
const struct HAL_USB_DEV g_usbdDev =
{
    .pReg = USB_OTG,
    .hclkGateID = HCLK_USBOTG_GATE,
    .utmiclkGateID = PCLK_USBPHY_GATE,
    .irqNum = OTG_IRQn,
    .BvalidIrqNum = OTG_BVALID_IRQn,
    .cfg =
    {
        .epNum = 10,  /* 支持的 USB IN/OUT 端点总数量 */
        .ep0Mps = USB_OTG_MAX_EP0_SIZE,
        .phyif = USB_PHY_UTMI_WIDTH_16,
        .speed = USB_OTG_SPEED_HIGH,
        .dmaEnable = true, /* 使能 USB DMA 传输 */
        .sofEnable = false,
        .lpmEnable = false,
        .vbusSensingEnable = false,
        .suspendEnable = true, /* 使能 USB Autosuspend 功能*/
    },
};
#endif
```

USB3.0 DRD Device BSP 参考配置如下：

```
#ifdef HAL_USB_DRD_MODULE_ENABLED
const struct HAL_USBDRD_DEV g_usbdrdDev =
{
    .pReg = USB3DRD,
    .irqNum = USB3OTG_IRQn,
    .busGateID = ACLK_USB3OTG0_GATE,
    .refGateID = CLK_REF_USB3OTG0_GATE,
    .suspendGateID = CLK_SUSPEND_USB3OTG0_GATE,
    .cfg =
    {
        .drMode = USB_DR_MODE_PERIPHERAL, /* 配置 DRD device mode */
        .hsphyMode = USBPHY_INTERFACE_MODE_UTMIW,
        .maximumSpeed = DRD_USB_SPEED_SUPER, /* 根据芯片设计和实际产品，配置支持的最大传
输速率 */
        .hsphyId = 0, /* 根据芯片设计，配置 USB2.0 PHY ID */
        .ssphyId = 1, /* 根据芯片设计，配置 USB3.0 PHY ID */
        .hasAwidth64 = 1, /* 使能 DMA 访问 64bit 总线地址 */
        .disDelPhyPowerChgQuirk = 1,
        .disEnblslpmQuirk = 1,
        .disRxdetInp3Quirk = 1,
        .disTxIpGapLinecheckQuirk = 1,
        .disU1u2Quirk = 1,
```

```
        .disU2SusphyQuirk = 1,
        .disU2FreeclkExistsQuirk = 1,
        .disU3SusphyQuirk = 1,
        .parkmodeDisableHsQuirk = 1,
        .parkmodeDisableSsQuirk = 1,
        .u2lpmDisable = 1,
    },
};
#endif
```

#### 4.1.1.1 USB Device VBUS Detect 使用说明

适用平台：RK2108/RK2116/RK2118/RK1820

支持三种不同的 VBUS Detect 方式

- GPIO VBUS Detect：通过 GPIO 检测 VBUS 电平

  对应宏定义 `USB_VBUS_PIN`

  参考 RK2118 evb 软硬件设计：`rt-thread/bsp/rockchip/rk2118/board/evb/board.h`
- PMIC VBUS Detect：通过 PMIC 检测 VBUS 电平

  对应宏定义 `RT_USING_USB_PMIC_VBUS`

  参考 RK2108 recording_pen 软硬件设计：`rt-thread/bsp/rockchip/rk2108/board/recording_pen_v10/board.h`
- USB PHY VBUS Detect：通过 USB PHY 检测 VBUS 电平

  对应宏定义 `RT_USING_BVALID_IRQ`

  参考 RK2118 iotest 软硬件设计：`rt-thread/bsp/rockchip/rk2118/board/iotest/board.h`

#### 4.1.1.2 USB Device speed 配置说明

适用平台：RK2108/RK2116/RK2118

USB Device 默认支持 High speed，如果产品上要求设备强制工作在 Full speed（如：UAC 产品形态），则可以配置

`#define RT_USING_USBD_SPEED_FULL`

#### 4.1.1.3 USB Device 端点 FIFO 配置说明

适用平台：RK2108/RK2116/RK2118

USB Device 端点 FIFO 是指 USB 控制器内部分配给各个端点的硬件 FIFO，所有 Rx 端点复用一个 RxFIFO，而每个 Tx 端点独占一个 TxFIFO。具体代码实现在 USB Device 控制器驱动适配层驱动以及板级配置中。

以 RK2118 SDK 为例：

USB 端点的板级默认配置在 `rt-thread/bsp/rockchip/rk2118/board/common/board_base.c`，如下所示，g_usb_ep_pool 按照大部分产品形态的 USB 功能进行设置，可同时支持 2 个 BULK 端点，2 个 ISOC 端点，2 个 INT 端点。如果产品上要支持其它 USB 复合设备，则需要根据实际所需的 USB 端点数量和类型，进行配置。

```
#ifdef RT_USING_USB_DEVICE
RT_WEAK struct ep_id g_usb_ep_pool[] =
{
    { 0x0,  USB_EP_ATTR_CONTROL,    USB_DIR_INOUT,  64,   ID_ASSIGNED   },
    { 0x1,  USB_EP_ATTR_BULK,       USB_DIR_IN,     1024, ID_UNASSIGNED },
    { 0x2,  USB_EP_ATTR_BULK,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
    { 0x3,  USB_EP_ATTR_ISOC,       USB_DIR_IN,     1024, ID_UNASSIGNED },
    { 0x4,  USB_EP_ATTR_ISOC,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
    { 0x5,  USB_EP_ATTR_INT,        USB_DIR_IN,     64,   ID_UNASSIGNED },
    { 0x6,  USB_EP_ATTR_INT,        USB_DIR_OUT,    64,   ID_UNASSIGNED },
    { 0xFF, USB_EP_ATTR_TYPE_MASK,  USB_DIR_MASK,   0,    ID_ASSIGNED   },
};
#endif
```

USB HAL 的 FIFO 设置在`rt-thread/bsp/rockchip/common/drivers/drv_usbd.c`

调用 `HAL_PCDEx_SetRxFiFo` 设置 RxFIFO，单位 4 字节

调用 `HAL_PCDEx_SetTxFiFo` 设置 TxFIFO，单位 4 字节

```
static void usb_pcd_lowlevel_hw_init(struct PCD_HANDLE *pcd)
{
    uint8_t i;

    /* Initialize Low Level Driver */
    HAL_PCD_Init(pcd);

#ifndef RT_USB_DEVICE_UVC
    HAL_PCDEx_SetRxFiFo(pcd, 0x118);
#else
    /*
     * Set smaller RxFiFo for 3 * UVC interfaces, then
     * it can assign enough TxFiFo size (1024 Bytes)
     * for each UVC streaming IN endpoint to get higher
     * performance.
     */
    HAL_PCDEx_SetRxFiFo(pcd, 0x8C);
#endif

    /* Set TxFIFOs according to the g_usb_ep_pool array */
    for (i = 0; i < pcd->cfg.epNum + 1; i++)
    {
        if (g_usb_ep_pool[i].addr == 0xFF)
            break;

        if (g_usb_ep_pool[i].dir == USB_DIR_IN || g_usb_ep_pool[i].dir ==
USB_DIR_INOUT)
            HAL_PCDEx_SetTxFiFo(pcd, g_usb_ep_pool[i].addr,
                                (g_usb_ep_pool[i].maxpacket >> 2));
    }
```

```
    HAL_PCD_Start(pcd);
}
```

### 4.1.2 RT-Thread 3.1.x/4.1.x USB Device 配置

USB Device 可以支持一个 interface 或者多个 interface 组合的复合设备（Composite Device），USB 组件允许开发者通过宏 `RT_USB_DEVICE_COMPOSITE` 开启复合功能。

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Using custom class by register interface)  --->
```

*Note*

1. `USB Vendor ID` 为 0x2207，是 USB-IF 官方授权给 Rockchip 的 VID，具有唯一性。USB-IF 规定 VID 为每个 Vendor 独有，同一个 VID 不能授权给不同的 Vendor 使用。因此 Rockchip USB VID 仅供产品开发和调试使用，如果产品需要过 USB-IF 认证，需要开发商自己申请 USB-IF 会员，以获取官方授权的唯一 Vendor ID；
2. `USB Product ID` 为 0x0007，可以根据实际产品来定义，规则是 0x00XX，保持高8位为0，避免与 Maskrom/Loader USB PID 冲突；
3. `Device type` 可以根据产品实际需求，进一步选择 CDC/MSC/HID/WINUSB/UAC1 等；
4. `Enable composite device` 用于支持 USB 复合设备，如：MSC + CDC-ACM 的组合。如果是单一 USB 功能，可以不用使能该选项；
5. USB Device 功能主要适用于 MCU 芯片，如：RK2108/RK2118/RK625。

### 4.1.3 RT-Thread 5.1.x USB Device 配置

根据产品 USB Device 需求，使能 USB 组件配置

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* 选择 USB Speed 自动探测 */
                  Select usb device ip (drd_rk)  /* 选择 USB Device 控制器 IP */
                  [ ]   Enable usb cdc acm device /* 选择 USB class 驱动 */
                  ...
                  [ ]   Enable usb dfu device (NEW)
                        Select usb device template ---> /* 选择 USB device demo 驱
动 */
                            ( ) cdc_acm
                            ( ) msc_ram
```

```
                                ...
                                ( ) winusbv2_hid
```

根据硬件设计，使能 USB 控制器驱动配置。以 RK1820 USB0 DRD Device 配置为例：

```
RT-Thread rockchip RK1820 drivers  --->
    Enable USB DRD Device  --->
        [*] Enable USB DRD Device controllers
            [*]   Enable USB DRD_0 Device
```

## 4.2 USB Host 配置

### 4.2.1 BSP USB Host 配置

USB2.0 OTG Host BSP 参考配置如下：

```
#ifdef HAL_HCD_MODULE_ENABLED
const struct HAL_USB_DEV g_usbotgh0Dev =
{
    .pReg = USB_OTG,
    .hclkGateID = HCLK_USBOTG_GATE,
    .utmiclkGateID = PCLK_USBPHY_GATE,
    .irqNum = OTG_IRQn,
    .BvalidIrqNum = OTG_BVALID_IRQn,
    .cfg =
    {
        .phyif = USB_PHY_UTMI_WIDTH_16,
        .speed = USB_OTG_SPEED_HIGH,
        .hcNum = 8, /* 支持的 USB Host channel 总数量 */
        .dmaEnable = true,
        .sofEnable = false,
        .lpmEnable = false,
        .suspendEnable = true, /* 使能 USB Autosuspend 功能 */
    },
};
#endif
```

USB2.0 DRD Host BSP 参考配置如下：

```
#ifdef HAL_USB_XHCI_MODULE_ENABLED
const struct HAL_USBXHCI_DEV g_usbxhciDev =
{
    .pReg = USB2DRD,
    .irqNum = USB2OTG_IRQn,
    .busGateID = ACLK_USB2OTG_GATE,
    .refGateID = CLK_REF_USB2OTG_GATE,
    .cfg =
    {
        .drMode = USB_DR_MODE_HOST, /* 配置 DRD Host mode */
        .hsphyMode = USBPHY_INTERFACE_MODE_UTMIW,
```

```
        .maximumSpeed = DRD_USB_SPEED_HIGH, /* 根据芯片设计和实际产品，配置支持的最大传
输速率 */
        .hsphyId = 0, /* 根据芯片设计，配置 USB2.0 PHY ID */
        .hasAwidth64 = 1, /* 使能 DMA 访问 64bit 总线地址 */
        .disDelPhyPowerChgQuirk = 1,
        .disEnblslpmQuirk = 1,
        .disRxdetInp3Quirk = 1,
        .disTxIpGapLinecheckQuirk = 1,
        .disU1u2Quirk = 1,
        .disU2SusphyQuirk = 1,
        .disU2FreeclkExistsQuirk = 1,
        .disU3SusphyQuirk = 1,
        .parkmodeDisableHsQuirk = 1,
        .parkmodeDisableSsQuirk = 1,
        .u2lpmDisable = 1,
    },
};
#endif
```

#### 4.2.1.1 USB Host VBUS 控制说明

适用平台：RK2116/RK2118/RK1820/RK3506/RK3568

支持通过 GPIO 控制 VBUS 输出 5V 的方式，在对应的板级头文件中添加 VBUS 对应的 GPIO BANK_PIN 宏定义即可。

1. USB2.0 OTG Host VBUS GPIO

   以 RK3506 EVB1 为例，VBUS 宏定义在 `bsp/rockchip/rk3506-32/board/evb1/board.h`

   ```
   #define USB_OTG_HOST0_VBUS_PIN        BANK_PIN(GPIO_BANK1, 20)
   #define USB_OTG_HOST1_VBUS_PIN        BANK_PIN(GPIO_BANK1, 24)
   ```

2. USB2.0 Host VBUS GPIO

   以 RK3568 EVB1 为例，VBUS 宏定义在 `bsp/rockchip/rk3568-32/board/rk3568_evb1/board.h`

   ```
   #define USB_HOST_VBUS_PIN        BANK_PIN(GPIO_BANK0, 6)
   ```

### 4.2.2 RT-Thread 3.1.x/4.1.x USB Host 配置

USB Host 目前仅支持 Mass Storage 功能，HUB 和 HID 功能正在开发中。

Mass Storage 功能的配置方法

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [*] Using USB host
            [*]   Enable Udisk Drivers
            (/)     Udisk mount dir
            [ ]   Enable HID Drivers
            [ ] Using USB device
            (4096) usb thread stack size
```

USB Host 驱动适配层配置项，位于 SoC drivers 路径下，以 RK2116 USB HOST 配置为例：

```
RT-Thread rockchip RK2116 drivers  --->
    Enable USB HOST  --->
        [*] Enable USB OTG HOST
        [*]   Enable USB OTG0 HOST
```

*Note*

1. USB Host 功能适用于 MCU 芯片和 AP 芯片。Host 驱动适配层的配置，需要根据实际芯片 USB 控制器的型号选择；
2. 由于 USB 驱动适配层的限制，不支持 `Using USB host` 和 `Using USB device` 同时使能，否则，会导致 USB 功能异常。

### 4.2.3 RT-Thread 5.1.x USB Host 配置

根据产品 USB Host 需求，使能 USB 组件配置

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb host mode  --->
                  Select usb host ip (xhci_rk)    /* 选择 USB Host 控制器 IP
xhci_rk */
                  [ ]   Enable usb cdc acm driver /* 根据产品需求，选择对应的 USB 设备
类驱动 */
                  ...
                  [ ]   Enable usb msc driver
                  [ ]   Use usb host template      /* USB Host Test Demo for test
acm/hid/... */
```

根据硬件设计，使能 USB 控制器驱动配置。以 RK1820 USB1 DRD Host 配置为例：

```
RT-Thread rockchip RK1820 drivers  --->
    Enable USB XHCI HOST  --->
        [*] Enable USB XHCI HOST controllers
        [*]   Enable USB XHCI1 HOST
```

# 5. USB 设备管理接口

USB 应用程序通过 RT-Thread 提供的 I/O 设备管理接口来访问 USB 硬件，相关接口如下所示：

表 2 USB Device 设备管理接口

| 函数 | 描述 |
| --- | --- |
| rt_device_find() | 查找 USB 设备 |
| rt_device_open() | 打开 USB 设备 |
| rt_device_read() | 读取 USB 数据 |
| rt_device_write() | 写入 USB 数据 |
| rt_device_control() | 控制 USB 设备 |

# 6. USB 内存管理接口

Rockchip 全系列 USB 控制器默认均使用 DMA 传输，因此，在执行 USB 读写操作时，需要保证数据 Buffer 起始地址按照 cacheline 对齐，且需要保证 DCache 的一致性。

**USB Date Buffer 的内存申请和释放接口**

建议优先使用 rt_malloc_align/rt_free_align。RT-Thread 常用的内存管理接口及功能描述如下表 3 所示。

表 3 RT-Thread 内存管理接口

| RTT malloc API | 功能描述 | 使用场景 |
| --- | --- | --- |
| rt_malloc<br>rt_free | 1. RTT 系统定义的API；<br>2. 从系统堆中分配内存；<br>3. 无法保证内存地址对齐；<br>4. 不保证uncache，在系统开启cache时，需要软件驱动刷Cache以保证Cache一致性。 | 在 RTT components 和 bsp/rockchip/common 中，申请非地址对齐和允许cache的内存时，广泛使用。 |
| rt_malloc_align<br>rt_free_align | 1. RTT 系统定义的API；<br>2. 从系统堆中分配内存；<br>3. 可保证内存地址对齐，且align size可配置，底层实现基于 rt_malloc API；<br>4. 不保证uncache，在系统开启cache时，需要软件驱动刷Cache以保证Cache一致性。 | 在 RTT components 和 bsp/rockchip/common 中，申请地址对齐和允许cache的内存时，广泛使用。<br>如：<br>components/drivers/usb/usbdevice/class/uac1.c<br>components/drivers/usb/usbdevice/class/hid.c<br>bsp/rockchip/common/drivers/drv_usbhost.c |
| rt_dma_malloc<br>rt_free_align | 1. RK自定义的API (依赖 RT_USING_DMA common/drivers/dma.c)；<br>2. 从在系统堆分配DMA内存；<br>3. 可保证内存地址基于CACHE_LINE_SIZE和DMA_ALIGN_SIZE对齐，底层实现基于 rt_malloc_align API；<br>4. 不保证uncache，在系统开启cache时，需要软件驱动刷Cache以保证Cache一致性。 | 在 bsp/rockchip/common中，明确使用SoC DMA 的模块中使用。<br>如：<br>bsp/rockchip/common/drivers/drv_spinand.c<br>bsp/rockchip/common/drivers/drv_sdio.c<br>bsp/rockchip/common/drivers/drv_pl330.c |
| rt_malloc_uncache<br>rt_free_uncache | 1. RK自定义的API (依赖 RT_USING_UNCACHE_HEAP common/drivers/drv_heap.c)；<br>2. 从指定的uncache堆分配DMA内存；<br>3. 无法保证内存地址对齐，底层实现基于 rt_memheap_alloc API；<br>4. 可保证uncache，在系统开启cache时，不需要软件驱动刷Cache来保证Cache一致性 | 1. 在 bsp/rockchip/common中，必须使用 uncache 内存的模块中使用。<br>如：<br>bsp/rockchip/common/drivers/drv_gmac.c<br>bsp/rockchip/common/drivers/drv_usbhost.c<br>2. 需要在系统启动时调用 rt_uncache_heap_init 初始化uncache堆<br>如：<br>rk3568-32/board/common/board_base.c |
| rt_dma_malloc_large<br>rt_dma_free_large | 1. RK自定义的API (依赖 RT_USING_UNCACHE_HEAP 和 RT_USING_LARGE_HEAP common/drivers/drv_heap.c)；<br>2. 从指定的large堆分配DMA内存；<br>3. 可保证内存地址基于CACHE_LINE_SIZE和DMA_ALIGN_SIZE对齐，底层实现基于 rt_memheap_alloc API；<br>4. 不保证uncache，在系统开启cache时，需要软件驱动刷Cache以保证Cache一致性。 | 1. 在 bsp/rockchip/common中，需要使用大块DMA内存的模块使用。<br>如：drv_display/drv_spi_screen/camera<br>2. 需要在系统启动时调用 rt_large_heap_init初始化large堆，目前只在RK2108/RK2206等极少数平台使用<br>如：rk2108/start/startup.c |

### USB Data Buffer 的 DCache Maintain 接口

1. RT-Thread 提供的 DCache Maintain 接口

   `void rt_hw_cpu_dcache_ops(int ops, void *addr, int size);`

   表 4 DCache Maintain 接口参数说明

| 参数 | 描述 |
| --- | --- |
| ops | 执行的 maintain 操作，RT_HW_CACHE_FLUSH 或者 RT_HW_CACHE_INVALIDATE |
| addr | 执行 maintain 操作的起始地址， |
| size | 执行 maintain 操作的大小 |

   使用注意点：maintain 操作的地址范围要覆盖所修改的数据，但不要超过，下面是一个具体例子：

```
buf = rt_dma_malloc(buf_size);
modify_start = &buf[50];
device_modify_data(modify_start, buf_size-50);

/* 注意起始地址不要用 buf，大小也不要用 buf_size，只要覆盖所修改的数据范围即可 */
rt_hw_cpu_dcache_ops(RT_HW_CACHE_INVALIDATE, modify_start, buf_size-50);
```

USB 驱动使用场景：`bsp/rockchip/common/drivers/drv_usbhost.c`

2. HAL 提供的 DCache Maintain 接口

```
HAL_DCACHE_CleanByRange(uintptr_t address, uint32_t sizeByte);
HAL_DCACHE_InvalidateByRange(uintptr_t address, uint32_t sizeByte);
HAL_DCACHE_CleanInvalidateByRange(uintptr_t address, uint32_t sizeByte);
```

USB 驱动使用场景:

```
lib/hal/src/hal_hcd.c
lib/hal/src/hal_pcd.c
lib/hal/src/hal_drd_gadget.c
```

# 7. USB Device 使用示例

## 7.1 RT-Thread 3.1.x/4.1.x USB Device 使用示例

### 7.1.1 USB Device 创建和注册

USB Device 通过 `rt_device_register()` 接口注册到 RT-Thread I/O 设备管理器中。

这里的 USB Device 是指 USB 控制器在 RT-Thread I/O 设备管理器中注册的设备，该设备只提供给 USB 设备类驱动（如HID/MStorage等）访问，应用程序无法直接访问 USB 控制器设备。此外，不同的 USB 设备类驱动，可能还会在 RT-Thread I/O 设备管理器创建和注册对应的设备，以方便应用程序访问 USB 设备，比如 `components/drivers/usb/usbdevice/class/hid.c` 通过 `rt_device_register` 接口创建 `hidd` 设备节点。

USB 控制器 Device 的创建和注册是在 RTT 与 HAL 的适配层 `bsp/rockchip/common/drivers/drv_usbd.c` 实现，流程如下图 5 所示。

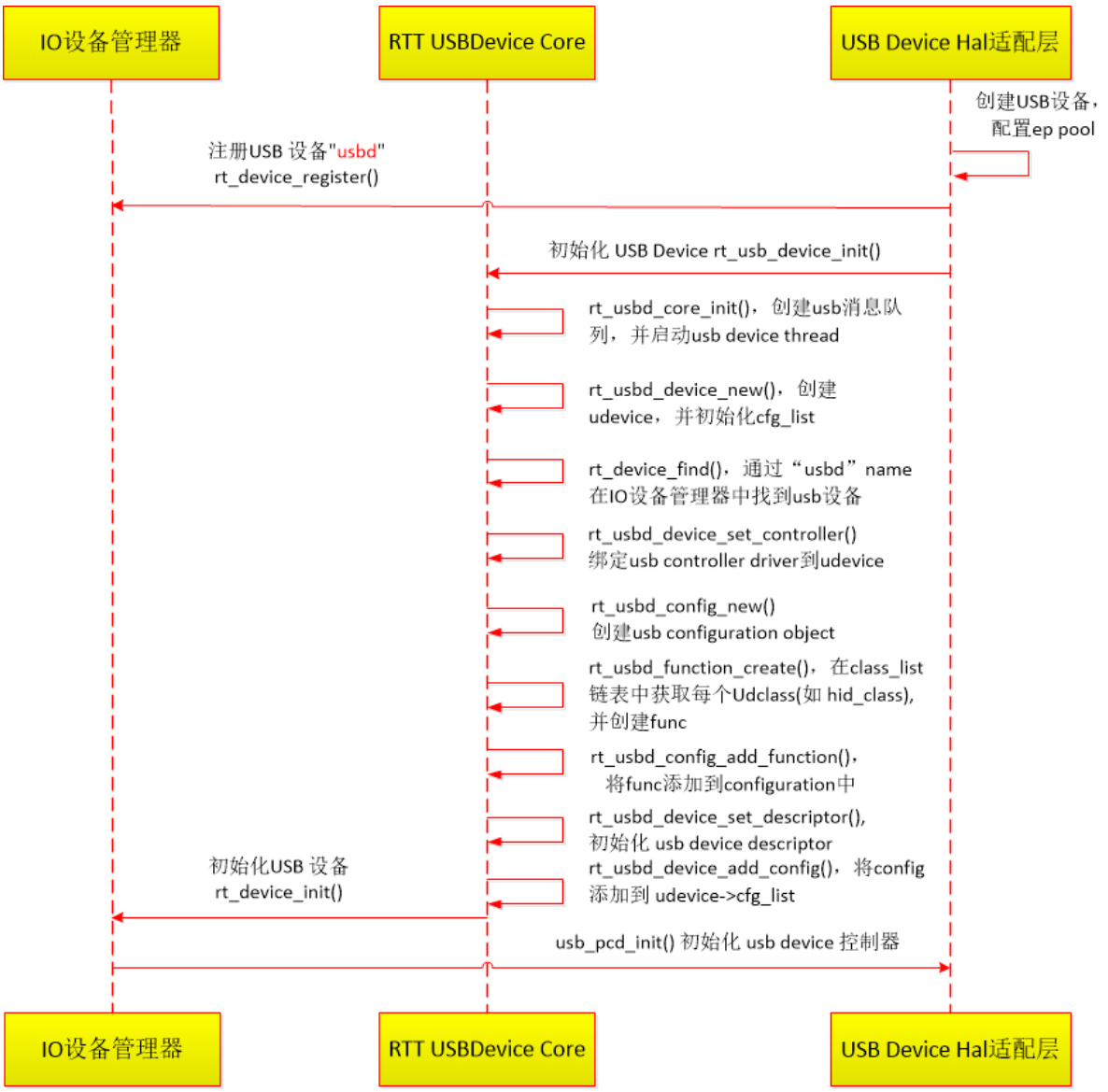



图 5 USB Device 的创建和注册流程

执行命令 `list_device` 可以看到已经生成的 USB Device 设备：

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
usbd     USB Slave Device     0
```

## 7.1.2 USB Device HID 使用示例

RT-Thread USB 组件支持常见的 USB Device HID 功能，包括：Keyboard, Mouse, General HID, media keyboard。用户可以根据实际的需求配置对应的 HID 功能。下面以 RK2108 USB HID Keyboard 为例，说明 HID 的配置和使用方法。

### USB Device HID 配置

执行命令 `scons --menuconfig`，将 USB 配置为 HID。

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
```

```
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                Device type (Enable to use device as HID device)  --->
            [*]   Use to HID device as Keyboard
            (1)     Number of Keyboard(max 3)
            [ ]   Use to HID device as Mouse
            [ ]   Use to HID device as General HID device
            [ ]   Use to HID device as media keyboard
```

执行命令 `list_device` 可以看到已经生成的 USB HID 设备，属于字符设备类型：

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
hidd     Character Device     0
```

**USB Device HID 的创建和注册流程**

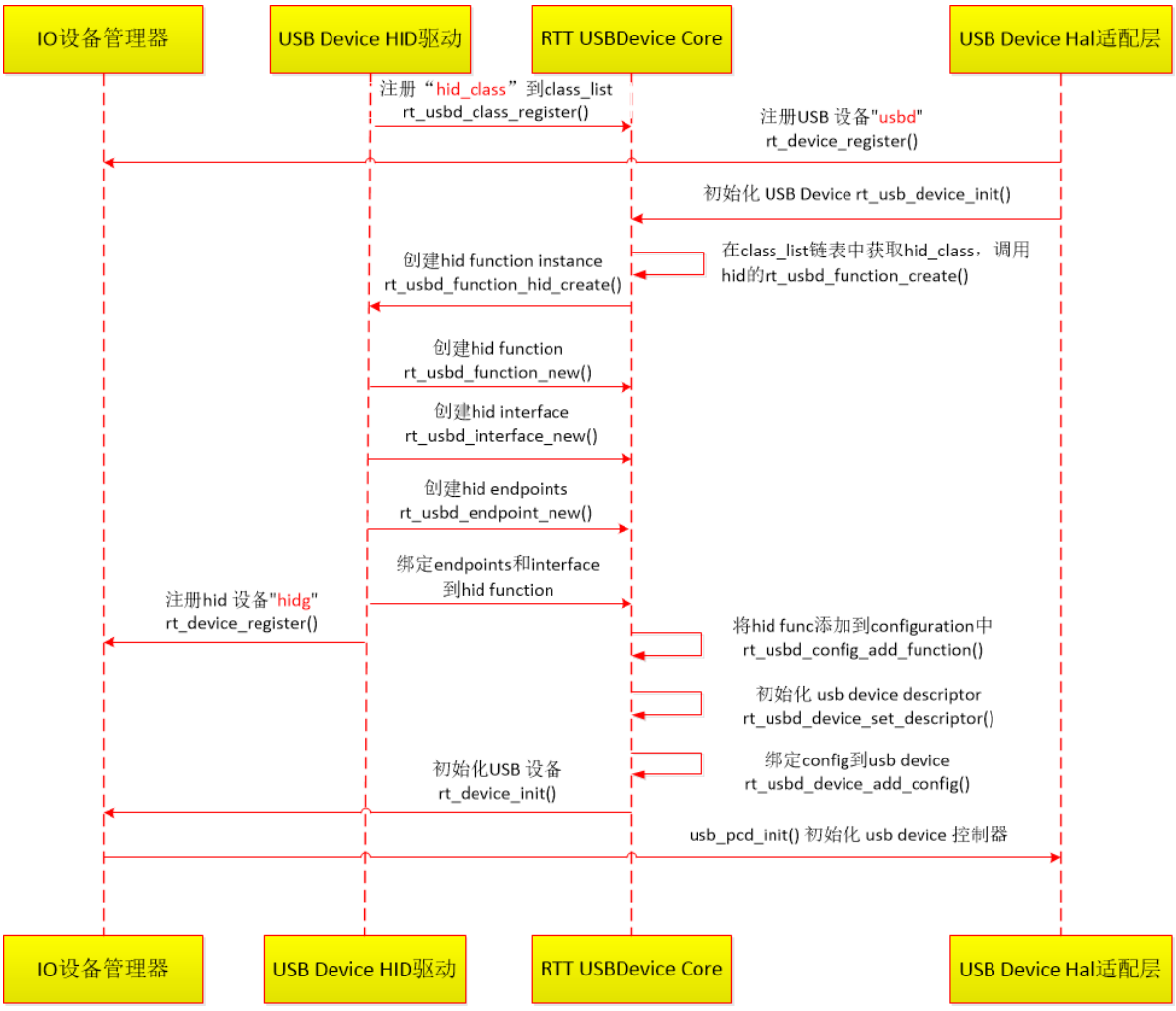



图 6 USB Device HID 的创建和注册流程

**应用程序访问 HID 设备的流程**

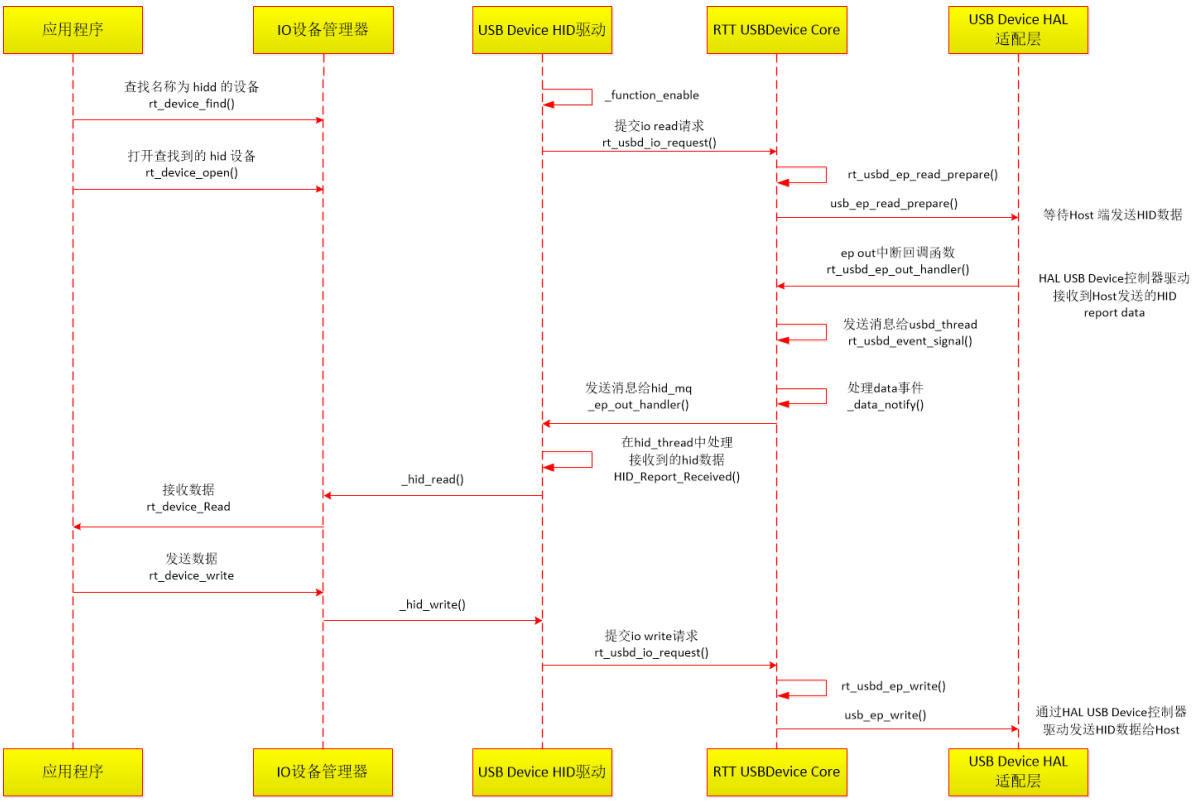



图 7 HID 应用程序访问设备的流程

### USB Device HID 应用开发参考用例

RT-Thread/IoT_Board 的 USB 鼠标例程

RT-Thread RK2108 HID 参考用例

### USB Device HID 传输性能优化

RT-Thread USB Device HID 受限于 HID 驱动的报告描述符以及 USB 控制器 INT 端点的 FIFO 配置，一次最大只能传输 64 bytes 的数据，传输性能比较差。可以参考章节 USB Device 端点 FIFO 配置说明 优化 HID 的传输性能，支持一次传输 1024 bytes 的数据。

以 RK2108 平台为例：

```
diff --git a/bsp/rockchip/rk2108/board/common/board_base.c
b/bsp/rockchip/rk2108/board/common/board_base.c
index 79cc20854..a494d7528 100644
--- a/bsp/rockchip/rk2108/board/common/board_base.c
+++ b/bsp/rockchip/rk2108/board/common/board_base.c
@@ -484,8 +484,8 @@ RT_WEAK struct ep_id g_usb_ep_pool[] =
     { 0x2,  USB_EP_ATTR_BULK,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
     { 0x3,  USB_EP_ATTR_ISOC,       USB_DIR_IN,     1024, ID_UNASSIGNED },
     { 0x4,  USB_EP_ATTR_ISOC,       USB_DIR_OUT,    512,  ID_UNASSIGNED },
-    { 0x5,  USB_EP_ATTR_INT,        USB_DIR_IN,     64,   ID_UNASSIGNED },
-    { 0x6,  USB_EP_ATTR_INT,        USB_DIR_OUT,    64,   ID_UNASSIGNED },
+    { 0x5,  USB_EP_ATTR_INT,        USB_DIR_IN,     1024,   ID_UNASSIGNED },
+    { 0x6,  USB_EP_ATTR_INT,        USB_DIR_OUT,    1024,   ID_UNASSIGNED },
     { 0xFF, USB_EP_ATTR_TYPE_MASK,  USB_DIR_MASK,   0,    ID_ASSIGNED   },
 };
 #endif
diff --git a/components/drivers/include/drivers/usb_common.h
b/components/drivers/include/drivers/usb_common.h
index d16b42ea7..2370cdc88 100644
```

```
--- a/components/drivers/include/drivers/usb_common.h
+++ b/components/drivers/include/drivers/usb_common.h
@@ -508,7 +508,7 @@ typedef struct uhid_descriptor* uhid_desc_t;
 struct hid_report
 {
     rt_uint8_t report_id;
-    rt_uint8_t report[63];
+    ALIGN(32) rt_uint8_t report[1024];
     rt_uint8_t size;
 };
 typedef struct hid_report* hid_report_t;
diff --git a/components/drivers/usb/usbdevice/class/hid.c
b/components/drivers/usb/usbdevice/class/hid.c
index f0e75178b..fdb12001e 100644
--- a/components/drivers/usb/usbdevice/class/hid.c
+++ b/components/drivers/usb/usbdevice/class/hid.c
@@ -194,14 +194,14 @@ const rt_uint8_t _report_desc[]=
     COLLECTION(1),      0x01,
     REPORT_ID(1),       HID_REPORT_ID_GENERAL,

-    REPORT_COUNT(1),    RT_USB_DEVICE_HID_GENERAL_IN_REPORT_LENGTH,
+    REPORT_COUNT(2),    0xff, 0x03,
     USAGE(1),           0x03,
     REPORT_SIZE(1),     0x08,
     LOGICAL_MINIMUM(1), 0x00,
     LOGICAL_MAXIMUM(1), 0xFF,
     INPUT(1),           0x02,

-    REPORT_COUNT(1),    RT_USB_DEVICE_HID_GENERAL_OUT_REPORT_LENGTH,
+    REPORT_COUNT(2),    0xff, 0x03,
     USAGE(1),           0x04,
     REPORT_SIZE(1),     0x08,
     LOGICAL_MINIMUM(1), 0x00,
@@ -347,7 +347,7 @@ const static struct uhid_comm_descriptor _hid_comm_desc =
         USB_DESC_TYPE_ENDPOINT,
         USB_DYNAMIC | USB_DIR_IN,
         USB_EP_ATTR_INT,
-        0x40,
+        1024,
         0x01,
     },

@@ -357,7 +357,7 @@ const static struct uhid_comm_descriptor _hid_comm_desc =
         USB_DESC_TYPE_ENDPOINT,
         USB_DYNAMIC | USB_DIR_OUT,
         USB_EP_ATTR_INT,
-        0x40,
+        1024,
         0x01,
     },
 };
@@ -607,7 +607,7 @@ static rt_size_t _hid_write(rt_device_t dev, rt_off_t pos,
const void *buffer, r
         rt_memcpy((void *)report.report,(void *)buffer,size);
         report.size = size;
```

```
         hiddev->ep_in->request.buffer = (void *)&report;
-        hiddev->ep_in->request.size = (size+1) > 64 ? 64 : size+1;
+        hiddev->ep_in->request.size = (size+1) > MAX_REPORT_SIZE ?
MAX_REPORT_SIZE : size+1;
         hiddev->ep_in->request.req_type = UIO_REQUEST_WRITE;
         rt_usbd_io_request(hiddev->func->device, hiddev->ep_in, &hiddev->ep_in-
>request);
         return size;
diff --git a/components/drivers/usb/usbdevice/class/hid.h
b/components/drivers/usb/usbdevice/class/hid.h
index 4d0f314a1..ea9938220 100644
--- a/components/drivers/usb/usbdevice/class/hid.h
+++ b/components/drivers/usb/usbdevice/class/hid.h
@@ -34,8 +34,8 @@ extern "C" {
 #define USB_HID_REQ_SET_IDLE        0x0a
 #define USB_HID_REQ_SET_PROTOCOL    0x0b

-#define MAX_REPORT_SIZE             64
-#define HID_RX_BUFSIZE              64
+#define MAX_REPORT_SIZE             1024
+#define HID_RX_BUFSIZE              1024

 /* HID Report Types */
 #define HID_REPORT_INPUT            0x01
```

### 7.1.3 USB Device ADB 使用示例

RT-Thread 官方提供了 ADBD（Android Debug Bridge daemon）软件包，支持通过 USB 或者 TCPIP 与 PC 之间进行文件传输或者执行 shell 的通道。

USB ADB 功能基于 WINUSB 驱动实现数据传输，ADBD 应用程序通过操作设备节点 `/dev/winusb`，可以与 USB 控制器进行数据交互。

ADBD 软件包的说明，请参考 RT-Thread 官网文档中心：https://packages.rt-thread.org/detail.html?package=adbd

ADBD 软件包的工程下载地址：https://github.com/heyuanjie87/adbd

**USB Device ADB 配置**

**配置 USB 组件**

ADB 的 WINUSB 设备类驱动在 ADBD 软件包 `packages/adbd-latest/core/adbusb.c` 中实现，因此，在 USB 组件中选择使用 custom class。

`USB Product ID` 建议配置为 0x0006，保持与 Rockchip AP 芯片平台 ADB 功能的 PID 一致，避免手动安装 PC ADB 驱动。

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            -*- Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0006) USB Product ID
            [ ]   Enable composite device
                  Device type (Using custom class by register interface)  --->
                      (X) Using custom class by register interface
```

**配置 ADBD online packages**

```
[*] Enable RT-Thread online packages  --->
    RT-Thread online packages  --->
        tools packages  --->
            [*] ADBD: Android Debug Bridge daemon implementation in RT-Thread  --
->
                [ ]   Using TCPIP transfer                  /* 启用 TCP/IP传输数据
*/
                [*]   Using USB transfer                    /* 启用 USB 传输数据
*/
                (1280) Set transfer thread stack size       /* 设置传输线程栈大小
*/
                [*]   Enable Shell service                  /* 开启 Shell 服务 */
                [*]   Enable File service                   /* 开启 文件 服务 */
                (2304)  Set file service thread stack size  /* 设置文件服务线程栈大
小 */
                (2000)  Set file service receive timeout(ms) /* 设置文件接收超时时间
*/
                [ ]   Enable external MOD                   /* 使能外部模块 */
                [ ]   Enable ADB service discover           /* 启用局域网发现服务
*/
                      Version (latest)  --->                /* 选择 ADBD 软件版本
*/
```

*Note*

`Using TCPIP transfer` 用于 TCPIP 传输方式，需要 disable，否则 ADBD 默认使用 TCPIP 传输方式，会影响 USB ADB 功能。

**配置 ADBD 依赖的 POSIX**

```
RT_USING_LEGACY [=y]
RT_USING_DFS [=y]
DFS_USING_POSIX [=y]
RT_USING_POSIX_FS [=y]
RT_USING_POSIX_DEVIO [=y]
RT_USING_POSIX_STDIO [=y]
RT_USING_POSIX_POLL [=y]
RT_USING_POSIX_SELECT [=y]
RT_USING_POSIX_SOCKET [=y]
```

**配置 ADB push/pull 文件依赖的文件系统**

启用文件系统的方法，请参考章节 USB Device MSC 使用示例 中关于 “USB Device MSC 文件系统自动挂载表” 的说明。

完成上述配置后，保存并退出配置界面。

**下载 ADBD online packages**

以 RK2118 SDK 为例：

```
~/rt-thread/bsp/rockchip/rk2118$ source ~/.env/env.sh
~/rt-thread/bsp/rockchip/rk2118$ pkgs --update
[Use Gitee server - auto decision based on IP location]
Cloning into '/home/wlf/rt-thread/bsp/rockchip/rk2118/packages/adbd-latest'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 42 (delta 2), reused 37 (delta 2), pack-reused 0
Receiving objects: 100% (42/42), 10.13 MiB | 1.74 MiB/s, done.
Resolving deltas: 100% (2/2), done.
==============================>  ADBD latest is downloaded successfully.

/home/wlf/rt-thread/bsp/rockchip/rk2118/packages/adbd-latest
==============================>  adbd update done

Operation completed successfully.
```

**修改 ADBD package 源码，以适配 Rockchip 平台**

```
diff --git a/core/adbusb.c b/core/adbusb.c
index 29455d0..034e9bb 100644
--- a/core/adbusb.c
+++ b/core/adbusb.c
@@ -232,7 +232,7 @@ static rt_err_t _ep_out_handler(ufunction_t func, rt_size_t
size)
 static rt_err_t _ep_in_handler(ufunction_t func, rt_size_t size)
 {
     winusb_device_t wd = (winusb_device_t)func->user_data;
-    wd->wrcnt -= size;
+    wd->wrcnt -= wd->ep_in->request.size;
     rt_wqueue_wakeup(&(wd->wq), (void *)POLLOUT);

     return RT_EOK;
@@ -284,13 +284,12 @@ static rt_err_t _interface_handler(ufunction_t func, ureq_t
setup)
     return RT_EOK;
 }

-#ifdef RT_USING_POSIX
 static int _ep_alloc_request(uep_t ep)
 {
     int size;

     size = EP_MAXPACKET(ep);
-    ep->buffer = rt_malloc(size);
+
```

```
+    ep->buffer = rt_malloc_align(RT_ALIGN(size, USB_DMA_ALIGN_SIZE),
USB_DMA_ALIGN_SIZE);
     if (!ep->buffer)
         return -1;

@@ -300,7 +299,6 @@ static int _ep_alloc_request(uep_t ep)

     return 0;
 }
-#endif

 static rt_err_t _function_enable(ufunction_t func)
 {
@@ -316,7 +314,7 @@ static rt_err_t _function_enable(ufunction_t func)
         return -1;
     if (_ep_alloc_request(wd->ep_in) != 0)
     {
-        rt_free(wd->ep_out->buffer);
+        rt_free_align(wd->ep_out->buffer);
         return -1;
     }
     _readreq(func, wd);
@@ -331,8 +329,8 @@ static rt_err_t _function_disable(ufunction_t func)
     RT_ASSERT(func != RT_NULL);
     wd = func->user_data;

-    rt_free(wd->ep_out->buffer);
-    rt_free(wd->ep_in->buffer);
+    rt_free_align(wd->ep_out->buffer);
+    rt_free_align(wd->ep_in->buffer);
     wd->ep_out->buffer = 0;
     wd->ep_in->buffer = 0;

diff --git a/services/shell_service.c b/services/shell_service.c
index 6252942..80e6cb0 100644
--- a/services/shell_service.c
+++ b/services/shell_service.c
@@ -344,6 +344,7 @@ static bool send_ready(struct shell_ext *ext, char *args)

 extern int libc_stdio_set_console(const char* device_name, int mode);
 extern int libc_stdio_get_console(void);
+extern int ioctl(int fildes, int cmd, ...);

 static int _shell_open(struct adb_service *ser, char *args)
 {
```

执行 `list_device` 和 `mount` 命令，查看 `winusb` 节点是否正常创建以及文件系统是否正常挂载。

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
winusb   Miscellaneous Device 0
usbd     USB Slave Device     0
root     Block Device         1
snor     MTD Device           0
```

```
fspi0    Character Device     0
i2c0     I2C Bus              0
pin      Pin Device           0
uart0    Character Device     2

msh />mount
filesystem  device  mountpoint
----------  ------  ----------
devfs       (NULL)  /dev
elm         root    /
```

**ADB 功能**

上位机的 ADB 工具和驱动在 ADBD packages 的 `adbd-latest/tools` 目录下。

支持 adb push/pull 文件

支持 adb shell（进入 adb shell 命令行后，Console 将从物理串口切换到 USB）

退出 adb shell 的命令：`exitas`

## 7.1.4 USB Device MSC 使用示例

USB MSC（Mass Storage Class）的功能，可以将设备作为 U 盘使用。当设备通过 USB 连接 PC 后，PC 可以识别到一个可移动的存储设备，方便设备和 PC 进行文件的拷贝。

以 RT-Thread RK2108 MSC 的使用为例

**USB Device MSC 配置**

执行命令 `scons --menuconfig`，将USB 配置为 Mass Storage device。

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Enable to use device as Mass Storage device)  --
->
            (root) msc class disk name
            (4096) msc buffer length
```

*Note*

1. 需要手动修改 `msc class disk name` 为 `root`
2. 需要使能 `CONFIG_RT_USING_DFS_MNTTABLE`
3. `msc buffer length` 用于优化 USB 控制器的传输性能，值越大，通常传输性能越好。该值不能低于 512，且不能大于 65536。

**USB Device MSC 文件系统自动挂载表**

USB Device MSC 功能依赖于文件系统挂载表。如果没有配置文件系统自动挂载表，可能会出现能枚举到设备但是无法读写的情况。

目前 SDK 支持 Littlefs 和 FAT/EXFAT 文件系统，如果产品需要文件系统，可以在 `menuconfig` 里把相应的文件系统打开，并在板级目录下新建一个文件 `mnt.c`，用来配置文件系统自动挂载表，下面是一个挂载表的配置示例：

配置文件系统

```
RT_USING_DFS [=y] #文件系统总开关
DFS_FILESYSTEMS_MAX [=2] #最大允许的挂载节点数目（即分区数目），因为devfs要占用一个，所以这个值要>=（需要挂载的物理分区+1）
DFS_FILESYSTEM_TYPES_MAX [=2] #最大允许的文件系统数目，支持elmfat和devfs，所以是2
DFS_FD_MAX [=16] #最大可同时操作的文件句柄
RT_USING_DFS_MNTTABLE [=y] #启用自动挂载表
RT_USING_DFS_ELMFAT [=y] #启用elmfat文件系统，兼容fat文件系统
RT_USING_DFS_DEVFS [=y] #启用设备文件系统，可以通过文件接口访问设备驱动
RT_DFS_ELM_MAX_SECTOR_SIZE [=4096] # 设置扇区大小为4096，以兼容底层spi flash的擦除块大小
```

参考 RK2108 EVB：`bsp/rockchip/rk2108/board/rk2108d_evb/mnt.c`

```
#ifdef RT_USING_DFS_MNTTABLE
#include <dfs_fs.h>

#define PARTITION_ROOT          "root"

/********************* Private Variable Definition ***************************/
/** @defgroup MNT_Private_Variable Private Variable
 *  @{
 */

/**
 * @brief  Config mount table of filesystem
 * @attention The mount_table must be terminated with NULL, and the partition's name
 * must be the same as above.
 */
const struct dfs_mount_tbl mount_table[] =
{
    {PARTITION_ROOT, "/", "elm", 0, 0},
#if defined(RT_USING_LITTLEFS) && defined(RT_USING_SPINAND)
    {"spinand0", "/nand", "lfs", 0, 0},
#elif defined(RT_USING_SDIO0) && defined(RT_SDCARD_MOUNT_POINT)
    {"sd0", RT_SDCARD_MOUNT_POINT, "elm", 0, 0},
#endif
    {0}
};
#endif
```

`dfs_mount_tbl` 结构体说明

```
/* mount table */
struct dfs_mount_tbl
{
    const char   *device_name;    /* 设备节点名 */
    const char   *path;           /* 挂载点目录 */
    const char   *filesystemtype; /* 文件系统名字(elm对应fat) */
    unsigned long rwflag;         /* rw(1为读写，0为只读，elm不支持这个标识，填0) */
    const void   *data;           /* 文件系统私有数据(elm不支持，填0) */
};
```

**USB Device MSC 的使用**

开机后，如果设备没有通过 USB 连接到 PC，`root` 分区默认挂载到设备端的文件系统上。此时，设备端可以对 `root` 分区进行读写。

当设备通过 USB 连接到 PC 后，USB 驱动会自动将 `root` 分区从设备端的文件系统上卸载掉，此时，设备端无法对 `root` 分区进行读写，而 PC 端可以自动识别到U盘，并可以对U盘进行读写。

当设备断开 USB 连接后，USB 驱动会自动卸载 `root` 分区，并重新挂载到设备端的文件系统上。

## 7.1.5 USB Device UAC1 使用示例

UAC（USB Audio Class）功能，可以将设备做为 PC 或其他上位机的扩展声卡使用，主要支持功能如下：

1. 支持 UAC 1.0 协议；
2. 支持 High speed（默认）和 Full speed，参考章节USB Device speed 配置说明；
3. 默认支持采样率 48 KHz，44.1 KHz，32 KHz，16 KHz，8 KHz；
4. 默认支持通道数 2 x playback channels 和 2 x capture channels，采样深度 16 Bits；
5. 默认支持 Volume 和 Mute 控制；
6. 支持独立使能录音和放音功能。

表 5 UAC1 主要宏定义说明

| UAC1 宏 | 默认值 (develop-v4.1.x) | 功能描述 |
| --- | --- | --- |
| SAMPLE_RATE_MAX | 48000 | 配置录音和放音的最大采样率，单位 Hz，最大支持 192000 |
| SAMPLE_RATE_MIN | 8000 | 配置录音和放音的最小采样率，单位 Hz，最小支持 8000 |
| SAMPLERATE_NUM | 5 | 配置录音和放音支持的总采样率数量 |
| P_NR_CHANNEL | 2 | 配置放音的通道数，最大支持 8 通道数 |
| P_DEFAULT_SAMPLE_RATE | 48000 | 配置放音默认采样率 |
| P_AUDIO_SAMPLE_BITS | 16 | 配置放音位深度，可配置 16/24/32 |
| P_PERIOD_SIZE_MS | 10 | 配置放音缓存的数据量，值越大则放音延时越大 |
| C_NR_CHANNEL | 2 | 配置录音的通道数，最大支持 8 通道数 |

| UAC1 宏 | 默认值 (develop-v4.1.x) | 功能描述 |
| --- | --- | --- |
| C_DEFAULT_SAMPLE_RATE | 48000 | 配置录音默认采样率 |
| C_AUDIO_SAMPLE_BITS | 16 | 配置录音位深度，可配置 16/24/32 |

### USB Device UAC1 配置（RT-Thread v3.1.x）

以 RK2108 SDK UAC1 配置为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC1 device)  --->
            (es8388p) uac1 playback card name
            (pdmc) uac1 capture card name
            [*] Enable usb audio playback function /* 使能UAC1放音功能 */
            [*] Enable usb audio record function /* 使能UAC1录音功能 */
            [ ] Enable to use pll compensation for clock synchronization /* 使能
PLL补偿，实现设备端的时钟动态调整 */
```

### USB Device UAC1 配置（RT-Thread v4.1.x）

以 RK2116/RK2118 SDK UAC1 配置为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC1 device)  --->
            (rk_dmy_playback_card) uac1 playback card name
            (rk_dmy_capture_card) uac1 capture card name
            [*] Enable usb audio playback function /* 使能UAC1放音功能 */
            [*] Enable usb audio record function /* 使能UAC1录音功能 */
            [ ] Enable to poll ep first transfer status /* 使能HAL层USB音频端点查询
方式替代中断方式，建议在复合设备时使能 */

RT-Thread board config  --->
    hifi4 rtt app  --->
        [*] uac support components
    [*] hifi4 rtt codecs  --->
        [*] using rk dummy audio card
```

### USB Device UAC1 + HID 复合设备的配置（RT-Thread v4.1.x）

以 RK2116/RK2118 SDK UAC1 + HID 配置为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0029) USB Product ID
            [*] Enable composite device
            [*] Enable to use device as HID device
            [*] Enable to use device as UAC1 device
            [*] Use to HID device as media keyboard
            (rk_dmy_playback_card) uac1 playback card name
            (rk_dmy_capture_card) uac1 capture card name
            [*] Enable usb audio playback function  /* 使能UAC1放音功能 */
            [*] Enable usb audio record function/* 使能UAC1录音功能 */
            [*] Enable to poll ep first transfer status /* 使能HAL层USB音频端点查询
方式替代中断方式，建议使能 */

RT-Thread board config  --->
    hifi4 rtt app  --->
        [*] uac support components
    [*] hifi4 rtt codecs  --->
        [*] using rk dummy audio card
```

*Note*

1. UAC1 功能开启时，必须关闭 VAD 功能，即设置 `# CONFIG_RT_USING_VAD is not set`；
2. `RT_USB_AUDIO_P_NAME` 对应实际的播放设备节点，需要根据实际的音频硬件设计进行修改。RK2108 SDK 参考设计配置为 `es8388p`，RK2116/RK2118 SDK 参考设计配置为 `rk_dmy_playback_card`；
3. `RT_USB_AUDIO_C_NAME` 对应实际的录音设备节点，需要根据实际的音频硬件设计进行修改。RK2108 SDK 参考设计配置为 `pdmc`，RK2116/RK2118 SDK 参考设计配置为 `rk_dmy_playback_card`；
4. `RT_USB_AUDIO_PLL_COMPENSATION` 用于使能 PLL 补偿，实现设备端的时钟动态调整。该功能用于设备放音场景，可以解决 USB 传输时钟（来自 USB HOST SOF）和 UAC1 设备端 I2S 时钟异源之间的时钟不同步，导致 UAC1 设备端出现 buffer overrun 或者 underrun 的问题。在使能该功能后，设备端能够主动调整自身数据处理的时钟，以适应 USB HOST 的数据发送率。这种方法的优点是，不依赖于 Windows/Linux/MAC OS 等USB 主机系统，具有更好的兼容性。缺点是：UAC1 设备端，需要根据不同 SoC 的 clock 设计方案，进行软件开发，实现时钟动态调整，有可能受限于芯片的 clock 设计。目前主要应用在 RK2108 芯片平台；
5. `RT_USB_AUDIO_USING_EP_POLL` 表示使用 HAL 层 USB 音频端点查询方式替代默认的中断方式，获取音频流启动传输的状态。如果产品上支持 UAC1 复合设备（如：UAC1 + HID），建议默认使能，可以避免端点中断风暴的潜在风险（与连接的 USB 主机行为有关），但同时，查询方式也会引入音频流延时的影响，SDK 默认设置查询间隔 `EP_STATUS_POLL_INTERVAL_MS` 为 5ms，如果产品对延时敏感，可以根据应用需求，修改查询间隔。
6. RT-Thread v3.1.x 的 UAC1 驱动默认不支持 USB 音频端点查询方式，可以通过更新驱动补丁的方式支持。

   补丁下载地址：https://redmine.rock-chips.com/attachments/1217535

系统启动后，如果找不到播放设备，PC 虽能能识别 UAC1 设备，但无法播放音频。而如果找不到录音设备，系统会出现 fault 出错。因此在打开 UAC1 功能之前，请使用 `list_device` 命令检查是否已经创建 Sound device。以 RK2108 SDK 为例，`es8311p` 对应音频播放设备，`pdmc` 对应音频录音设备。

如果未发现音频播放和录音设备，请在 menuconfig 中打开相应的 Audio 编译开关。

```
msh />list_device
device          type         ref count
------- -------------------- ----------
es8311c Sound Device         0
es8311p Sound Device         1
audpwmp Sound Device         0
pdmc    Sound Device         0
adcc    Sound Device         0
usbd    USB Slave Device     0
root    Block Device         1
snor    MTD Device           0
pwm0    Miscellaneous Device 0
dmac0   Miscellaneous Device 0
i2c0    I2C Bus              0
pin     Miscellaneous Device 0
uart2   Character Device     0
uart0   Character Device     2
```

### 7.1.6 USB Device UAC2 使用示例

UAC（USB Audio Class）功能，可以将设备做为 PC 或其他上位机的扩展声卡使用，主要支持功能如下：

1. 支持 UAC 2.0 协议；
2. 支持 High speed（默认）和 Full speed，参考章节USB Device speed 配置说明；
3. 默认支持放音采样率 192KHz，96KHz，48 KHz，44.1 KHz；
4. 默认支持录音采样率 96KHz，48 KHz，44.1 KHz；
5. 默认支持通道数 2 x playback channels 和 2 x capture channels，采样深度 16 Bits；
6. 默认支持 Volume 和 Mute 控制；
7. 支持独立使能录音和放音功能；
8. 不支持 3KB/微帧的高带宽传输方式。

表 6 UAC2 主要宏定义说明

| UAC2 宏 | 默认值 (develop-v4.1.x) | 功能描述 |
| --- | --- | --- |
| P_NR_CHANNEL | 2 | 配置放音的通道数，最大支持 16 通道数 |
| P_AUDIO_SAMPLE_BITS | 16 | 配置放音位深度，可配置 16/24/32 |
| P_PERIOD_SIZE_MS | 5 | 配置放音缓存的数据量，值越大则放音延时越大 |
| P_SAMPLE_RATE_DEFAULT | 48000 | 配置放音默认采样率 |
| C_NR_CHANNEL | 2 | 配置录音的通道数，最大支持 16 通道数 |

| UAC2 宏 | 默认值 (develop-v4.1.x) | 功能描述 |
| --- | --- | --- |
| C_AUDIO_SAMPLE_BITS | 16 | 配置录音位深度，可配置 16/24/32 |
| C_SAMPLE_RATE_DEFAULT | 48000 | 配置录音默认采样率 |
| UAC2_HS_INTERVAL_DEFAULT | 0x04 | 配置 HS USB 音频帧间隔，默认 1ms (2^(4-1) = 8 uframes) |
| UAC2_FS_INTERVAL_DEFAULT | 0x01 | 配置 FS USB 音频帧间隔，固定为 1ms |

### USB Device UAC2 配置（RT-Thread v3.1.x）

以 RK2108 SDK UAC2 配置为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC2 device)  --->
            (es8388p) uac playback card name
            (pdmc) uac capture card name
            [*] Enable usb audio playback function /* 使能UAC1放音功能 */
            [*] Enable usb audio record function /* 使能UAC1录音功能 */
```

### USB Device UAC2 配置（RT-Thread v4.1.x）

以 RK2116/RK2118 SDK UAC2 配置为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0019) USB Product ID
            [ ] Enable composite device
                  Device type (Enable to use device as UAC2 device)  --->
            (rk_dmy_playback_card) uac1 playback card name
            (rk_dmy_capture_card) uac1 capture card name
            [*] Enable usb audio playback function /* 使能UAC1放音功能 */
            [*] Enable usb audio record function /* 使能UAC1录音功能 */
            [ ] Enable to poll ep first transfer status /* 使能HAL层USB音频端点查询
方式替代中断方式，建议在复合设备时使能 */

RT-Thread board config  --->
    hifi4 rtt app  --->
        [*] uac support components
    [*] hifi4 rtt codecs  --->
        [*] using rk dummy audio card
```

## 7.1.7 USB Device UVC 使用示例

UVC（USB Video Class）功能，可以将设备做为 USB 视频采集设备使用，Rockchip RT-Thread UVC 基于 UVC 1.1 标准协议实现，支持 MJPEG 和 YUYV 两种格式视频数据，目前最多支持三路 UVC 复合设备。

UVC 使用限制：

1. 仅支持 BULK 传输方式，不支持 ISOC 传输方式；
2. 仅支持 RK625 芯片平台。

**USB Device UVC 配置**

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [*]   Enable composite device
            [*]     Enable to use device as UVC device
            [*]   Use to UVC device as RGB
            [ ]   Use to UVC device as DEPTH
            [ ]   Use to UVC device as IR
            (10240) uvc device max buffer length
```

*Note*

1. RGB，DEPTH，IR 需要根据具体 SoC 的硬件支持情况进行配置。如：RK625 SoC 可支持 RGB UVC，RK625 + RK1608 方案可支持三路 UVC；
2. `uvc device max buffer length` 可支持 4K-16K 范围配置。

**UVC 设计简述**

UVC 组件基于 RT-Thread USB Framework 实现，主要包括 UVC 线程、UVC 驱动、Buffer 管理和异常处理等子模块，其系统框图如下所示。

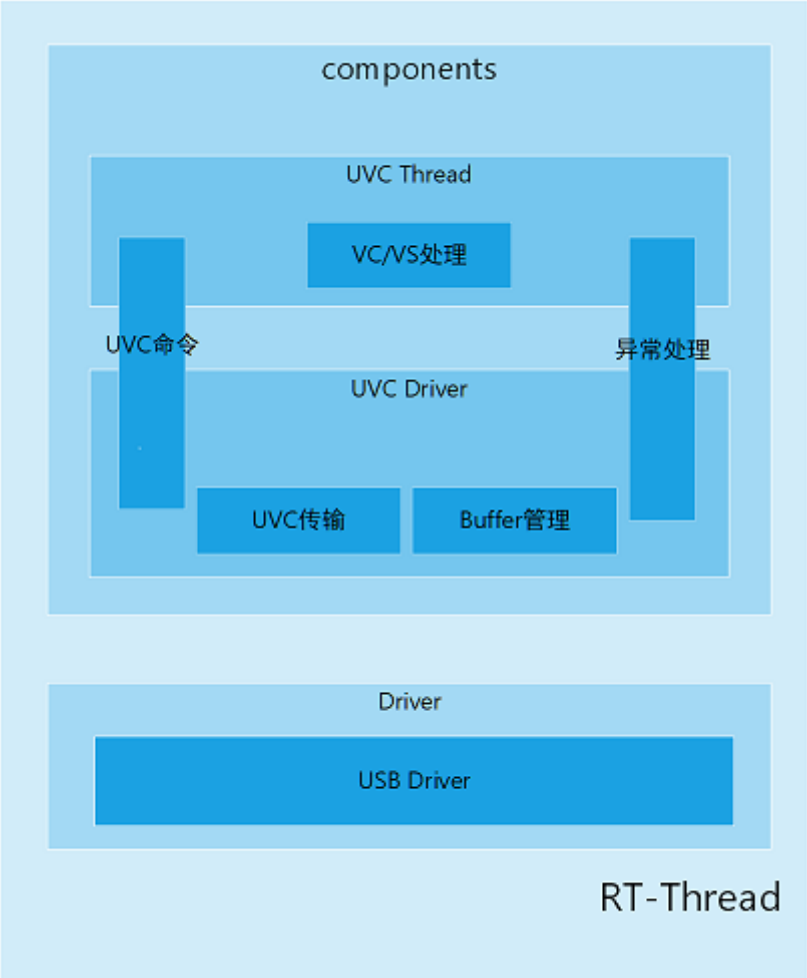



图 8 UVC 组件框图

- UVC Thread 主要实现 UVC VC/VS 请求响应，每路 UVC 有单独一个处理线程；
- UVC Driver 实现 UVC 设备描述符定义（包括图像分辨率定义）、UVC Buffer 状态管理、UVC 视频数据传输调度和异常处理等功能；
- UVC Thread 和 UVC Driver 通过 RT-Thread OS Event 通信；UVC Driver 和 USB Driver 通过 USB IO 接口通信。

**UVC 日志解析**

将 UVC 的设备接入上位机，并在上位机上使用视频预览软件打开 Camera，Rockchip SoC 串口在上电 Camera 初始化后，将有如下 UVC 相关 Log 输出：

```
[...1...]
INF: uvc(1) set format fcc 1196444237 width 640 height 480
VIDEO_OPEN(1)
INF: uvc(1) open camera device...
Open camera id width height 1 640 480
[...2...]
INF: uvc(1) buffer(0) addr 0x2006787c
INF: uvc(1) buffer(1) addr 0x2006987c
INF: uvc(1) buffer(2) addr 0x2006b87c
INF: uvc(1) buffer(3) addr 0x2006d87c
INF: uvc(1) mjpeg header addr 0x20087c00
INF: uvc(1) buffer size 8064
[...]
[...3...]
VIDEO_CLOSE(1)
INF: uvc(1) close camera device...
```

```
Close camera id 1
close rgb success
[...]
[...4...]
uvc(1) buffer: (0-1) (1-1) (2-1) (3-2)
ERR: uvc(1) buffer overflow irq 0x000100f0
```

- Log 1 指示被打开 Camera 的 FCC 及宽高等信息；
- Log 2 指示 UVC 环形 Buffer 的使用个数，地址和每个 Buffer 的大小；如果是 MJPEG 格式的数据，还会打印 MJPEG Header Buffer 信息；
- 在 Log 1 和 Log 2 成功输出后，如果 Sensor 有送数据出来，UVC 将会调度 Buffer 管理并传送数据给上位机；
- Log 3 指示上位机关闭 Camera 时的 Log；
- Log 4 指示诸如 AMCAP，GUVCView 等常用的视频预览软件在关闭预览或退出时不会发送 Close Camera 命令，只是向 USB 发送一条 EP Halt 命令。EP Halt 后，会触发 UVC Buffer Overflow，Buffer 管理器继而关闭 Buffer通道，停止 USB 数据传输。

### 7.1.8 USB Device WINUSB 使用示例

WINUSB 是 Windows 操作系统提供的一种通用 USB 驱动程序，无需安装驱动即可实现 Bulk 数据传输。

**USB Device WINUSB 配置**

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Enable to use device as winusb device)  --->
            ({6860DC3C-C05F-4807-8807-1CA861CC1D66}) Guid for winusb
            (16384) winusb transfer length
```

执行命令 `list_device` 可以看到已经生成的 USB WINUSB 设备：

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
winusb   Miscellaneous Device 0
usbd     USB Slave Device     0
```

针对 WINUSB 功能，Rockchip 提供了一个上位机的 Demo，可以用于 WINUSB 功能测试。

下载路径：https://redmine.rock-chips.com/attachments/1967928

## 7.1.9 USB Device CDC_VCOM 使用示例

CDC_VCOM 支持 USB 作为虚拟串口与上位机进行数据收发，基于 CDC-ACM 协议实现驱动。

**USB Device CDC_VCOM 配置**

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
            [ ] Using USB host
            [*] Using USB device
            (4096) usb thread stack size
            (0x2207) USB Vendor ID
            (0x0007) USB Product ID
            [ ]   Enable composite device
                    Device type (Enable to use device as CDC device)  --->
            (512) virtual com thread stack size
            (128) virtual com rx buffer size
            [ ]   Enable to use dma for vcom tx
            (320219198301O8) serial number of virtual co
            (14)  serial number length of virtual com
            (1000) tx timeout(ticks) of virtual com
```

**RT-Thread 3.1.x 实现 USB system console 的修改方法**

```
diff --git a/components/libc/compilers/newlib/libc.c
b/components/libc/compilers/newlib/libc.c
index 2234c36f98..07ba2bafeb 100644
--- a/components/libc/compilers/newlib/libc.c
+++ b/components/libc/compilers/newlib/libc.c
@@ -23,6 +23,7 @@ int   _EXFUN(putenv,(char *__string));

 int libc_system_init(void)
 {
+    rt_console_set_device("vcom");
 #if defined(RT_USING_DFS) & defined(RT_USING_DFS_DEVFS) &
defined(RT_USING_CONSOLE)
     rt_device_t dev_console;

@@ -47,4 +48,4 @@ int libc_system_init(void)

     return 0;
 }
-INIT_COMPONENT_EXPORT(libc_system_init);
+INIT_APP_EXPORT(libc_system_init);
```

**RT-Thread 4.1.x 实现 USB system console 的修改方法**

支持 console 动态切换：

- USB 连接 Host，并且 Host 端通过串口工具（如：Microcom/Minicom/SecureCRT）打开 USB 串口设备，console 自动切换到 USB VCOM
- USB 断开与 Host 的连接，console 自动切换到原来系统启动默认的 console

依赖 POSIX 配置

```
RT_USING_POSIX_FS [=y]
RT_USING_POSIX_DEVIO [=y]
RT_USING_POSIX_STDIO [=y]
```

```
diff --git a/components/drivers/usb/usbdevice/class/cdc_vcom.c
b/components/drivers/usb/usbdevice/class/cdc_vcom.c
index ba70a04b27..9de44a5041 100644
--- a/components/drivers/usb/usbdevice/class/cdc_vcom.c
+++ b/components/drivers/usb/usbdevice/class/cdc_vcom.c
@@ -82,6 +82,7 @@ struct vcom
     rt_uint8_t tx_rbp[CDC_TX_BUFSIZE];
     struct rt_ringbuffer tx_ringbuffer;
     struct rt_event  tx_event;
+    rt_device_t old_ser_device;
 };

 struct vcom_tx_msg
@@ -464,8 +465,18 @@ static rt_err_t _interface_handler(ufunction_t func, ureq_t
setup)
         break;
     case CDC_SET_CONTROL_LINE_STATE:
         data->connected = (setup->wValue & 0x01) > 0?RT_TRUE:RT_FALSE;
-        RT_DEBUG_LOG(RT_DEBUG_USB, ("vcom state:%d \n", data->connected));
+        rt_kprintf("usb vcom connect state: %d \n", data->connected);
         dcd_ep0_send_status(func->device->dcd);
+        if (data->connected)
+        {
+            if (data->old_ser_device)
+            {
+                rt_kprintf("switch old console %s to usb vcom.\n",
+                           data->old_ser_device->parent.name);
+            }
+            rt_console_set_device("vcom");
+            finsh_set_device("vcom");
+        }
         break;
     case CDC_SEND_BREAK:
         break;
@@ -525,6 +536,12 @@ static rt_err_t _function_disable(ufunction_t func)
     _vcom_reset_state(func);

     data = (struct vcom*)func->user_data;
+    if (data->old_ser_device)
+    {
+        rt_console_set_device(data->old_ser_device->parent.name);
+        finsh_set_device(data->old_ser_device->parent.name);
+    }
+
     if(data->ep_out->buffer != RT_NULL)
     {
         rt_free_align(data->ep_out->buffer);
@@ -958,6 +975,8 @@ static void rt_usb_vcom_init(struct ufunction *func)
```

```
     data->serial.serial_rx  = RT_NULL;
     data->serial.config     = config;

+    data->old_ser_device = rt_console_get_device();
+
     /* register vcom device */
     rt_hw_serial_register(&data->serial, VCOM_DEVICE,
 #ifndef VCOM_TX_USE_DMA
@@ -987,4 +1006,41 @@ int rt_usbd_vcom_class_register(void)
 }
 INIT_PREV_EXPORT(rt_usbd_vcom_class_register);

+int usb_vcom_enable(void)
+{
+    rt_device_t cdc_vcom;
+    rt_device_t old_device;
+
+    cdc_vcom = rt_device_find("vcom");
+    if (cdc_vcom == RT_NULL)
+    {
+        rt_kprintf("not found usb vcom device!\n");
+        return -RT_ERROR;
+    }
+
+    old_device = rt_console_get_device();
+    if (old_device)
+    {
+        rt_kprintf("switch old console %s to usb vcom.\n",
+                   old_device->parent.name);
+    }
+
+    rt_console_set_device(cdc_vcom->parent.name);
+    finsh_set_device(cdc_vcom->parent.name);
+
+    return RT_EOK;
+}
+MSH_CMD_EXPORT(usb_vcom_enable, enable usb cdc virtual console);
+
+int usb_vcom_disable(void)
+{
+    /* Switch to default console device */
+    rt_kprintf("switch usb vcom to %s.\n", RT_CONSOLE_DEVICE_NAME);
+    rt_console_set_device(RT_CONSOLE_DEVICE_NAME);
+    finsh_set_device(RT_CONSOLE_DEVICE_NAME);
+
+    return RT_EOK;
+}
+MSH_CMD_EXPORT(usb_vcom_disable, disable usb cdc virtual console);
+
 #endif
```

**CDC_VCOM 兼容性问题处理**

问题描述： 通过 USB 虚拟串口和上位机进行通信，Rockchip Device 端可以接收上位机发送的串口消息，但是 Device 端无法正常发送串口消息给上位机。

解决方法：

```
Subject: [PATCH] [COMPONENTS]: drivers: usb: cdc_vcom: Fix connected state

The current cdc vcom driver depends the setup wValue of
the request CDC_SET_CONTROL_LINE_STATE to set the connected
state. Refer to CDC Spec V1.2 Chapter 6.3.12 SetControlLineState,
the Bit0 of the wValue is used to indicates to DCE if DTE
(Data Terminal Equipment, for example, a Personal Computer)
is present or not. However, some DTE always set the Bit0 of
the wValue to 0, this cause cdc vcom failed to send message to
DTE. This patch ignores the Bit0 of wValue, and set the
connected state to true directly if it receives the request
CDC_SET_CONTROL_LINE_STATE.

diff --git a/components/drivers/usb/usbdevice/class/cdc_vcom.c
b/components/drivers/usb/usbdevice/class/cdc_vcom.c
index df5e03f54c..e201742747 100644
--- a/components/drivers/usb/usbdevice/class/cdc_vcom.c
+++ b/components/drivers/usb/usbdevice/class/cdc_vcom.c
@@ -463,7 +463,7 @@ static rt_err_t _interface_handler(ufunction_t func, ureq_t
setup)
         _cdc_get_line_coding(func->device, setup);
         break;
     case CDC_SET_CONTROL_LINE_STATE:
-        data->connected = (setup->wValue & 0x01) > 0?RT_TRUE:RT_FALSE;
+        data->connected = RT_TRUE;
         RT_DEBUG_LOG(RT_DEBUG_USB, ("vcom state:%d \n", data->connected));
         dcd_ep0_send_status(func->device->dcd);
         break;
```

## 7.2 RT-Thread 5.1.x USB Device 使用示例

### 7.2.1 USB Device ADB 使用示例

RT-Thread 5.1.x ADB 基于 RT-Thread 官方提供的 ADBD（Android Debug Bridge daemon）软件包并做了改进，支持 USB3.0 传输，同时优化了 PUSH/PULL 的传输速率，源码集成到了 `rt-thread/third_party/adbd` 目录下，不再需要下载 ADBD online packages。

以 RK1820 USB ADB 使用为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* 选择 USB Speed 自动探测 */
                  Select usb device ip (drd_rk)  /* 选择 USB Device 控制器 IP */
                  Select usb device template (none (Implement it yourself))

RT-Thread third party package  --->
     [*] ADBD: Android Debug Bridge daemon implementation in RT-Thread  --->
         [ ]   Using TCPIP transfer
```

```
        [*]   Using USB transfer
        (4096) Set transfer thread stack size
        [*]   Enable Shell service
        [*]   Enable File service
        (4096)  Set file service thread stack size
        (5000)  Set file service receive timeout(ms)
        [ ]   Enable external MOD
        [ ]   Enable ADB service discovery

RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

*Note*

ADB 驱动需要申请的 buffer size 比较大，建议 board config 选项中的 Memory size 至少为 2MB。

执行命令 `list_device` 可以看到已经生成的 USB ADB 设备

```
msh />list device
device           type         ref count
-------- -------------------- ----------
usb_adb  Miscellaneous Device 0
```

**ADB 功能**

上位机的 ADB 工具和驱动在 ADBD packages 的 `adbd-latest/tools` 目录下。

支持 adb push/pull 文件

支持 adb shell（进入 adb shell 命令行后，Console 将从物理串口切换到 USB）

退出 adb shell 的命令：`exitas`

## 7.2.2 USB Device WINUSB 使用示例

CherryUSB Device 协议栈当前支持 WINUSB1.0、WINUSB2.0、WEBUSB 功能。以 RK1820 WINUSB 使用为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* 选择 USB Speed 自动探测 */
                  Select usb device ip (drd_rk)  /* 选择 USB Device 控制器 IP */
                  Select usb device template (winusbv1_rt) /* 选择  WINUSB1.0 RTT
驱动 */
                      (winusb0) usb device winusb interface0 name
                      [ ]       Enable usb device with double winusb /* 选择支持两
个独立的 winusb interface */

RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
```

```
        [*]   Enable USB DRD_0 Device
```

执行命令 `list_device` 可以看到已经生成的 USB WINUSB 设备：

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
winusb0  Miscellaneous Device 0
```

WINUSB 测试用例参考

```
/**
  * Copyright (c) 2025 Fuzhou Rockchip Electronics Co., Ltd
  *
  * SPDX-License-Identifier: Apache-2.0
  */
#include <rtthread.h>
#include <rtdevice.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <sys/stat.h>
#include <stdio.h>
#include <math.h>
#include "drv_heap.h"

typedef struct utc_d_client_s
{
    rt_device_t device;
    rt_uint8_t *rev_buffer;
    rt_uint8_t *snd_buffer;
    rt_uint32_t package_len;
    rt_sem_t rx_notice;
    rt_sem_t tx_notice;
    rt_uint8_t send_busy_flag;
} utc_d_client_t;

static utc_d_client_t utc_d_client;

static rt_err_t utc_d_rx_ind(rt_device_t dev, rt_size_t size)
{
    utc_d_client_t *client = &utc_d_client;

    /*read the usb data next */
    rt_device_read(dev, 0, client->rev_buffer, 512);
    if (size)
    {
        rt_kprintf("utc_d_rx_ind rev size = %d \n", size);
    }
    return RT_EOK;
}

static rt_err_t utc_d_tx_ind(rt_device_t dev, void *buffer)
{
```

```
    utc_d_client_t *client = &utc_d_client;

    rt_kprintf("utc_d_tx_ind snd_buffer = %p \n", buffer);
    client->send_busy_flag = 0;

    return RT_EOK;
}

static void winusb_thread(void *p)
{
    utc_d_client_t *client = &utc_d_client;

    while (1)
    {
        if (!client->send_busy_flag)
        {
            client->send_busy_flag = 1;
            rt_device_write(client->device, 0, client->snd_buffer, 512);
        }
        rt_thread_mdelay(10);
    }
}

int winusb_app_init(void)
{
    utc_d_client_t *client = &utc_d_client;
    client->device = rt_device_find("winusb");
    rt_err_t result;

    if (client->device)
    {
        rt_kprintf("find winusb device\n");
        /* using the tx interrupt when uart is RS485 */
        result = rt_device_open(client->device, RT_DEVICE_OFLAG_RDWR);
        RT_ASSERT(result == RT_EOK);
        client->rx_notice = rt_sem_create("winusb_rx", 0, RT_IPC_FLAG_FIFO);
        client->tx_notice = rt_sem_create("winusb_tx", 0, RT_IPC_FLAG_FIFO);

        client->rev_buffer = (rt_uint8_t *)rt_malloc_align(512, 64);
        client->snd_buffer = (rt_uint8_t *)rt_malloc_align(512, 64);
        rt_memset(client->rev_buffer, 0, 512);
        rt_memset(client->snd_buffer, 0x31, 512);

        client->send_busy_flag = 0;

        rt_device_set_tx_complete(client->device, utc_d_tx_ind);
        rt_device_set_rx_indicate(client->device, utc_d_rx_ind);

        /*wait read the winusb device success */
        while (rt_device_read(client->device, 0, client->rev_buffer, 512) == 0)
        {
            rt_thread_mdelay(500);
        }
    }
    else
```

```
    {
        rt_kprintf("Not find winus device\n");
        return RT_ERROR;
    }

    rt_thread_t thread;

    thread = rt_thread_create("clockdark", winusb_thread, RT_NULL, 2048, 5, 10);
    RT_ASSERT(thread != RT_NULL);

    rt_thread_startup(thread);
    return RT_EOK;
}

INIT_APP_EXPORT(winusb_app_init);
```

针对 WINUSB 功能，Rockchip 提供了一个上位机的 Demo，可以用于 WINUSB 功能测试。

下载路径：https://redmine.rock-chips.com/attachments/1967928

### 7.2.3 USB Device CDC_ACM 使用示例

CherryUSB Device 协议栈当前支持 Communication Device Class (CDC_ACM)，以 RK1820 CDC_ACM 使用为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* 选择 USB Speed 自动探测 */
                  Select usb device ip (drd_rk)  /* 选择 USB Device 控制器 IP */
                  [*]   Enable usb cdc acm device /* 选择 CDC ACM Class 驱动 */
                  Select usb device template (cdc_acm) /* 选择 CDC ACM Device 驱动
*/

RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

*Note*

SDK 提供的 CDC_ACM DEMO 驱动 `components/drivers/usb/cherryusb/demo/cdc_acm_template.c` 没有在 RT-Thread I/O 设备管理器创建和注册对应的 ACM 设备，如果产品上需要通过应用程序访问 ACM 设备，需要通过 `rt_device_register` 接口创建 ACM 设备节点，可参考 SDK WINUSB&CDC_ACM 复合设备驱动的代码实现 `components/drivers/usb/cherryusb/demo/winusb1.0_cdc_acm_rt.c`。

## 7.2.4 USB Device CDC_VCOM 使用示例

CDC_VCOM 支持 USB 作为虚拟串口与上位机进行数据收发，基于 CDC-ACM 协议实现驱动，可以通过 USB 输出 console 信息，以及进行控制台命令交互。以 RK1820 CDC_VCOM 使用为例：

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* 选择 USB Speed 自动探测 */
                  Select usb device ip (drd_rk)  /* 选择 USB Device 控制器 IP */
                  -*-   Enable usb cdc acm device /* 选择 CDC ACM Class 驱动 */
                  -*-     Enable usb cdc acm as virtual com /* 配置 CDC VCOM 参数
*/
                  (1024) virtual com thread stack size
                  (128) virtual com rx buffer size
                  [ ]   Enable to use dma for vcom tx
                  (1000) tx timeout(ticks) of virtual com
                      Select usb device template -->
                          (X) cdc_acm_vcom /* 选择 CDC VCOM Device 驱动 */
RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

#### 7.2.4.1 USB Virtual Console切换方式

方式1.

- USB 连接 Host，并且 Host 端通过串口工具（如：Microcom/Minicom/SecureCRT）打开 USB 串口设备，console 自动切换到 USB VCOM
- USB 断开与 Host 的连接，console 自动切换到原来系统启动默认的 console

方式2.

- 在系统启动时默认的 console（通常为 RT_CONSOLE_DEVICE_NAME），通过如下的控制台命令切换到 USB VCOM

  `msh /> usb_vcom_enable`
- 在 USB VCOM 的控制台，通过如下命令切换到系统启动默认的 console

  `msh /> usb_vcom_disable`

## 7.2.5 USB Device WINUSB&CDC_ACM 复合设备使用示例

支持 1 x WINUSB Interface + 1 x CDC_ACM Interface + 1 x CDC_VCOM Interface

```
RT-Thread Components  --->
    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*]   Enable usb device mode  --->
                  Select usb device speed (AUTO) /* 选择 USB Speed 自动探测 */
```

```
                    Select usb device ip (drd_rk)  /* 选择 USB Device 控制器 IP */
                    -*-   Enable usb cdc acm device /* 选择 CDC ACM Class 驱动 */
                    [*]     Enable usb cdc acm as virtual com /* 选择 CDC VCOM Class
驱动 */
                    (1024) virtual com thread stack size
                    (128) virtual com rx buffer size
                    [ ]   Enable to use dma for vcom tx
                    (1000) tx timeout(ticks) of virtual com
                        Select usb device template -->
                            (X) (X) winusbv1_cdc_acm_rt /* 选择 WINUSB&CDC_ACM
Device 驱动 */
RT-Thread rockchip RK1820 drivers  --->
     Enable USB DRD Device  --->
         [*] Enable USB DRD Device controllers
         [*]   Enable USB DRD_0 Device
```

执行命令 `list device` 可以看到已经生成的 USB WINUSB&CDC_ACM 复合设备：

```
device           type         ref count
-------- -------------------- ----------
acm      Character Device     0
winusb0  Character Device     0
usb_vcom Character Device     0
```

在 RK1820 芯片平台上，各个 Interface 的功能描述如下：

表 7 WINUSB&CDC_ACM Interface 功能描述

| Interface | Interface Name | 功能描述 |
| --- | --- | --- |
| WINUSB | NTB Interface | 用于传输 RK1820 NPU 数据 |
| CDC_ACM | SMI Interface[Note1] | 用于传输 RK1820 SMI 信息 |
| CDC_VCOM | VCOM Interface[Note2] | 用于传输 RK1820 console 信息和控制台命令交互 |

*Note1*. 在 Linux USB Host 端，可以通过查询 /sys/class/tty/ttyACM*/device/interface 找到对应 SMI Interface。

*Note2*. 在 Linux USB Host 端，可以通过查询 /sys/class/tty/ttyACM*/device/interface 找到对应 VCOM Interface。USB Virtual Console 的使用方法参考章节USB Virtual Console切换方式。

# 8. USB Host 使用示例

## 8.1 RT-Thread 3.1.x/4.1.x USB Host 使用示例

### 8.1.1 USB Host 创建和注册

USB Host 通过 `rt_device_register()` 接口注册到 RT-Thread I/O 设备管理器中。通过创建一个 USB Hub 线程实现对端口的实时监听，当端口状态出现变化时枚举设备。同时，通过初始化一个设备类驱动链表，将不同的驱动添加到链表中实现对各种设备类驱动的管理。

RT-Thread 支持三种不同的 USB HOST 控制器类型的驱动适配层，分别是：DWC2/EHCI/OHCI，对应的代码路径请参考章节 USB 代码路径

以控制器 DWC2 的创建和注册为例，流程如下图 9 所示。

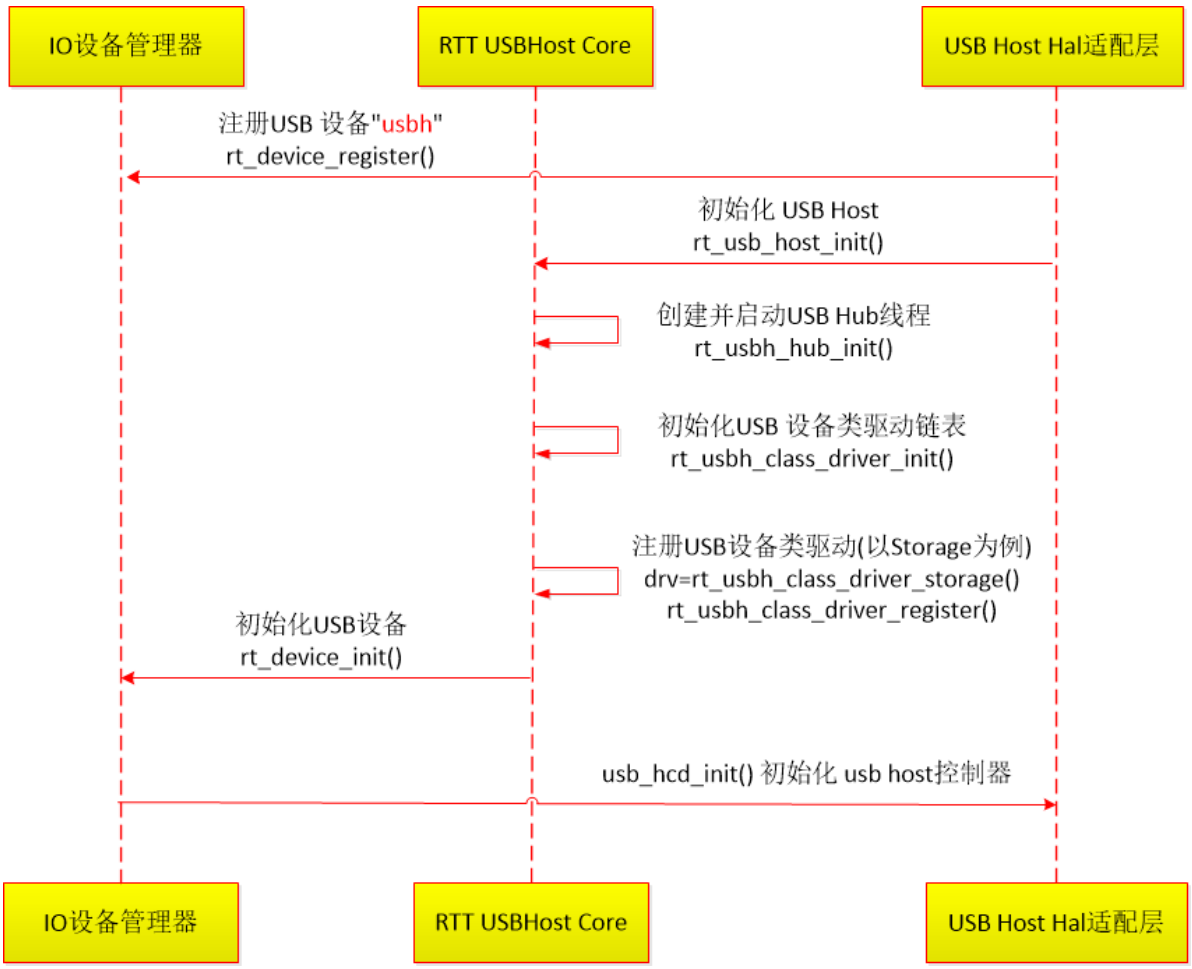



图 9 USB Host 的创建和注册流程

*Note*

如果控制器类型为 EHCI/OHCI，对应的驱动适配层文件是 `bsp/rockchip/common/drivers/drv_usbhost.c`，创建和注册流程与 DWC2 控制器基本一致。

执行命令 `list_device` 可以看到已经生成的 USB Host 设备：

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
usbh     USB Host Bus         0
```

## 8.1.2 USB Host MSC 使用示例

### USB Host MSC 组件配置

```
RT-Thread Components  --->
    Device Drivers  --->
        Using USB  --->
        [*] Using USB host
        [*]   Enable Udisk Drivers         /* 选择 Udisk 驱动 */
        (/udisk)     Udisk mount dir (NEW) /* 设置 Udisk 挂载目录名，可修改 */
        [ ]   Enable HID Drivers (NEW)
        [ ] Using USB device
        (4096) usb thread stack size       /* 设置 usb host 线程堆栈，建议保持默认值或
者适当增加 */
```

### USB Host 驱动适配层配置

RT-Thread 3.1.x USB Host 驱动适配层配置

需要根据芯片平台实际支持的 USB 控制器型号和 RT-Thread 软件版本进行配置。

```
RT-Thread rockchip common drivers  --->
    RT-Thread rockchip USB Host driver  --->
        [ ] DWC2 HCD Support
        [ ] EHCI HCD Support
        [ ] OHCI HCD Support
```

*Note*

1. `DWC2 HCD Support` 对应 DWC2 控制器驱动适配层；
2. `EHCI HCD Support` 和 `OHCI HCD Support` 对应 USB2 Host EHCI & OHCI 控制器驱动适配层；

RT-Thread 4.1.x USB Host 驱动适配层配置

为了在不同芯片平台上支持多 USB Host 接口同时工作的功能，从 RT-Thread v4.1.x SDK 开始，USB Host 驱动适配层的配置从平台通用驱动移到芯片驱动中。

以 RK3506 芯片平台同时支持 OTG0/1 两个 Host 接口为例，配置如下：

```
RT-Thread rockchip RK3506 drivers  --->
    Enable USB HOST  --->
        [*] Enable USB OTG HOST
        [*]   Enable USB OTG0 HOST
        [*]   Enable USB OTG1 HOST
```

### USB Host MSC 功能依赖的 FAT 文件系统相关配置

```
RT-Thread Components  --->
    [*] DFS: device virtual file system  ----
        [*] Enable elm-chan fatfs
              elm-chan's FatFs, Generic FAT Filesystem Module  --->
                ...
                (4) Number of volumes (logical drives) to be used. /* 表示支持的
Drives/Volumes 的总数量，默认值为 2 */
                (4096) Maximum sector size to be handled. /* 表示支持的最大扇区大小，
默认值为 512 */
```

*Note*

1. 默认只支持 FAT-32 格式的 Udisk，如果要支持 exFAT 格式，需要增加 `#define RT_DFS_ELM_USE_EXFAT`
2. 配置项 `Number of volumes (logical drives) to be used` 支持的 Drives/Volumes 总数量包括：U盘挂载的分区，以及通过 mount table 挂载的分区（如：root 分区和 data 分区）。在产品开发时，需要根据实际支持的分区数量正确配置该参数；
3. 支持 Udisk 自动挂载功能，但只支持单分区挂载，如果 Udisk 是多分区，只挂载第一个分区；
4. 支持 Udisk Application 通过 rt_device_control() 标准接口控制 Udisk 的卸载，具体方法参考表 4。

#### USB Host MSC 功能依赖的 cache 配置

```
RT-Thread rockchip common drivers  --->
    [*] Enable cache
```

#### USB Host MSC 设备管理接口

表 8 RT-Thread 4.1.x Udisk 设备管理接口

| 函数 | 描述 |
| --- | --- |
| rt_udisk_read() | 读取 Udisk 数据<br>支持 POSIX 文件操作接口 read和 C 标准库函数 fread |
| rt_udisk_write() | 写入 Udisk 数据<br>支持 POSIX 文件操作接口 write 和 C 标准库函数 fwrite |
| rt_udisk_control() | 控制 Udisk 卸载<br>RT_USBH_CLASS_CTRL_ENABLE：使能控制 Udisk 卸载的权限<br>RT_USBH_CLASS_CTRL_CONNECTED：查询 Udisk 连接状态<br>RT_USBH_CLASS_CTRL_STOP：执行 Udisk 卸载的操作 |

#### USB Host MSC 注册和枚举流程

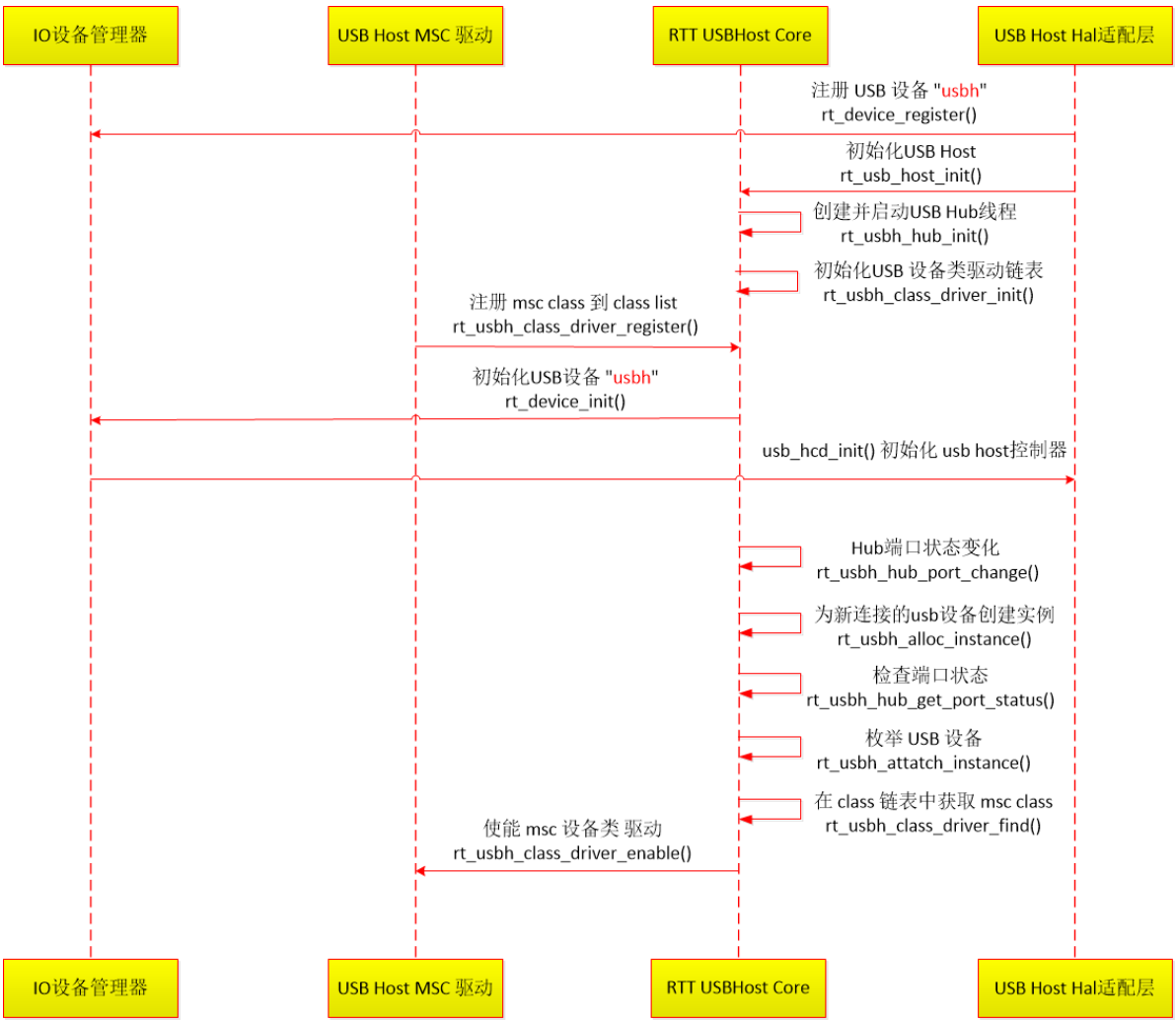



图 10 USB Host MSC 注册和枚举流程

**应用程序访问MSC设备流程**

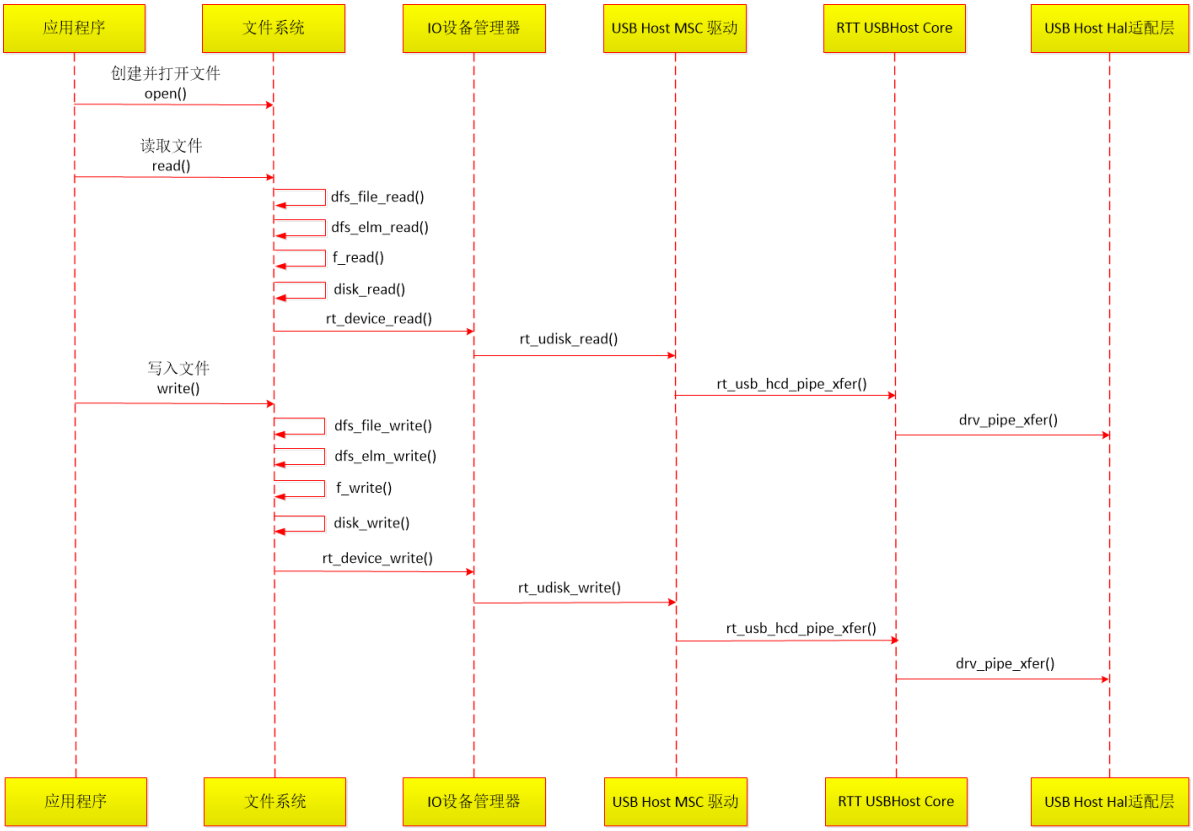



图 11 MSC 应用程序访问设备的流程

## 8.2 RT-Thread 5.1.x USB Host 使用示例

RT-thread 5.1.x CherryUSB Host 目前只支持 xHCI 控制器，传输方式上支持 Control 传输和 Bulk 传输。

### 8.2.1 USB Host MSC 使用示例

CherryUSB Host MSC 驱动支持 Udisk 的自动挂载和读写操作，以 RK1820 USB1 Host MSC 使用为例：

```
RT-Thread Components  --->
    DFS: device virtual file system  --->
        [*] Enable elm-chan fatfs
              elm-chan's FatFs, Generic FAT Filesystem Module  --->
                ...
                (4) Number of volumes (logical drives) to be used. /* 表示支持的
Drives/Volumes 的总数量，默认值为 2 */
                (4096) Maximum sector size to be handled. /* 表示支持的最大扇区大小，
默认值为 512 */

    Device Drivers  --->
        [*] Using USB with CherryUSB  --->
            [*] Enable usb host mode  --->
                  Select usb host ip (xhci_rk)    /* 选择 USB Host 控制器 IP
xhci_rk */
                  ...
                  [*] Enable usb msc driver       /* 选择 Udisk 驱动 */
                  (/udisk) Udisk mount dir        /* 设置 Udisk 挂载目录名，可修改
*/

RT-Thread rockchip RK1820 drivers  --->
    Enable USB XHCI HOST  --->
        [*] Enable USB XHCI HOST controllers
        [*] Enable USB XHCI1 HOST
```

*Note*

1. 默认只支持 FAT-32 格式的 Udisk，如果要支持 exFAT 格式，需要增加 `#define RT_DFS_ELM_USE_EXFAT`
2. 配置项 `Number of volumes (logical drives) to be used` 支持的 Drives/Volumes 总数量包括：U盘挂载的分区，以及通过 mount table 挂载的分区（如：root 分区和 data 分区）。在产品开发时，需要根据实际支持的分区数量正确配置该参数；
3. 支持 Udisk 自动挂载功能，但只支持单分区挂载，如果 Udisk 是多分区，只挂载第一个分区；
4. 支持 Udisk Application 通过 rt_device_control() 标准接口控制 Udisk 的卸载，具体方法参考表 5。

**USB Host MSC 设备管理接口**

表 9 RT-Thread 5.1.x Udisk 设备管理接口

| 函数 | 描述 |
| --- | --- |
| rt_udisk_read() | 读取 Udisk 数据<br>支持 POSIX 文件操作接口 read和 C 标准库函数 fread |
| rt_udisk_write() | 写入 Udisk 数据<br>支持 POSIX 文件操作接口 write 和 C 标准库函数 fwrite |
| rt_udisk_control() | 控制 Udisk 卸载<br>RT_CHERRY_USBH_CTRL_ENABLE：使能控制 Udisk 卸载的权限<br>RT_CHERRY_USBH_CTRL_CONNECTED：查询 Udisk 连接状态<br>RT_CHERRY_USBH_CTRL_STOP：执行 Udisk 卸载的操作 |